MODULE 11

Python Performance Tuning and Optimization

Measure, understand and improve the speed and resource use of Python programs

```
>>> timeit.timeit("sorted(data)", globals=globals())
```

AIN3005 · Advanced Python Programming

Binnur Kurt, PhD · Bahçeşehir University · AI Engineering



Learning Objectives

By the end of this module, you will be able to:

1. Define performance metrics such as latency percentiles, throughput, utilization and scalability
2. Apply Amdahl's law and Little's law to estimate speed-up limits and concurrency
3. Follow a systematic process: monitor, profile, then tune against measurable requirements
4. Explain how caches, the TLB, branch prediction and context switches affect program speed
5. Diagnose CPU, memory, disk and network bottlenecks with Linux and Windows tools
6. Measure Python code with `timeit`, `cProfile`, `tracemalloc` and sampling profilers
7. Optimize Python code with suitable data structures, built-ins, caching and laziness
8. Decide when to go beyond pure Python: processes, NumPy, Cython, Numba or PyPy

MODULE 11 · PERFORMANCE TUNING

Module Roadmap

01 **Application Performance**
What “fast” means and how to quantify it

02 **Performance Methodology**
Monitor, profile, tune — in that order

03 **Hardware and OS Fundamentals**
Why the machine behaves the way it does

04 **Monitoring CPU, Memory, Disk and Network**
Reading the symptoms with Linux and Windows tools

05 **Measuring Python Code**
Benchmark to know how fast; profile to know where the time goes

06 **Pure-Python Optimizations**
Better algorithms first, then cheaper operations

07 **Beyond Pure Python**
Processes, vectorisation, compilers and batching

01

Application Performance

What “fast” means and how to quantify it

- Performance metrics: from footprint to degradation
- Latency distributions, percentiles and Little's law
- CPython memory management and pauses
- Amdahl's law: the limits of parallel speed-up

What Is Performance?

Speed

How long does one piece of work take?

- Latency
- Responsiveness
- Startup time

Volume

How much work gets done?

- Throughput
- Capacity

Resource use

What does the work cost?

- Memory footprint
- Utilization
- Efficiency

Behaviour under load

What happens as load grows?

- Scalability
- Degradation

Key idea

Performance is not one number. Each aspect becomes an explicit, prioritized requirement for *your* application.

Memory Footprint

- **Memory footprint**: memory used by your process — interpreter, objects, native libraries
- Dedicated machine, or shared with other processes and containers?
- Footprint > physical RAM → **paging** to disk, orders of magnitude slower
- Container limit exceeded → the process is killed (out-of-memory)

object sizes in bytes (64-bit CPython)

```
>>> import sys
>>> sys.getsizeof(42)
28
>>> sys.getsizeof(3.14)
24
>>> sys.getsizeof("")
41
>>> sys.getsizeof([])
56
>>> sys.getsizeof({})
64
```

Note

Every Python object carries a header; a list stores 8-byte pointers to separate objects.

Startup Time

- **Startup time**: from launch until the application is ready for work
- Critical for CLIs, serverless functions and autoscaled containers
- Less important for long-running servers
- In Python, **imports** usually dominate: measure with `-X importtime`
- **Time to performance**: warm caches, connection pools, specialized bytecode

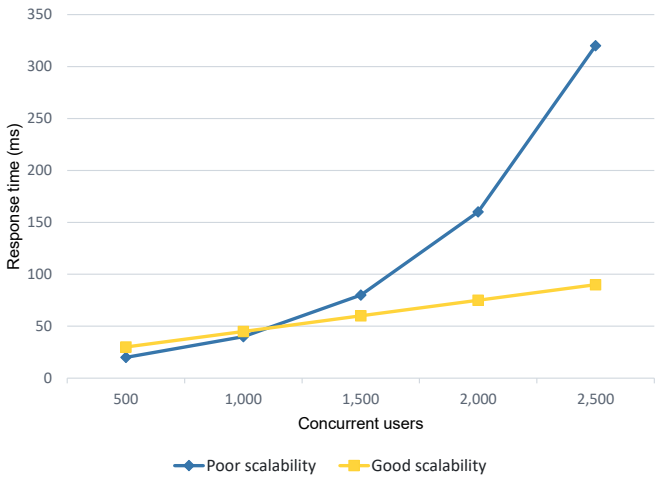
TERMINAL — SAMPLE OUTPUT (TIMINGS VARY BY MACHINE)

```
$ python -X importtime -c "import json"
import time: self [us] | cumulative | imported package
...
import time:       966 |       2327 |         collections
import time:       865 |       3253 |       functools
import time:       607 |       7733 |     re
import time:       511 |       8730 |   json.decoder
import time:       477 |        477 |   json.encoder
import time:       343 |       9549 | json
```

> **Note**
> Importing `json` took 9.5 ms here, most of it inside `re` and `collections`.

Scalability

- **Scalability**: how performance changes as load increases
- Good: response time grows roughly **linearly**
- Poor: response time grows **exponentially** until the system collapses
- Fast in development ≠ fast in production: test with realistic load
- Measured at every level: function, process, host, cluster



Responsiveness vs Throughput

	Responsiveness	Throughput
Question	How quickly is *one* request answered?	How much work is done per unit of time?
Examples	UI reacts to a click; web page returns; query result arrives	Jobs per hour; queries per second; records per minute
Long pauses	Unacceptable	Acceptable if the total is high
Typical systems	Interactive apps, APIs, games	Batch jobs, ETL pipelines, model training
Typical metric	Latency percentiles (p95, p99)	Requests/s, records/h, MB/s

Key idea

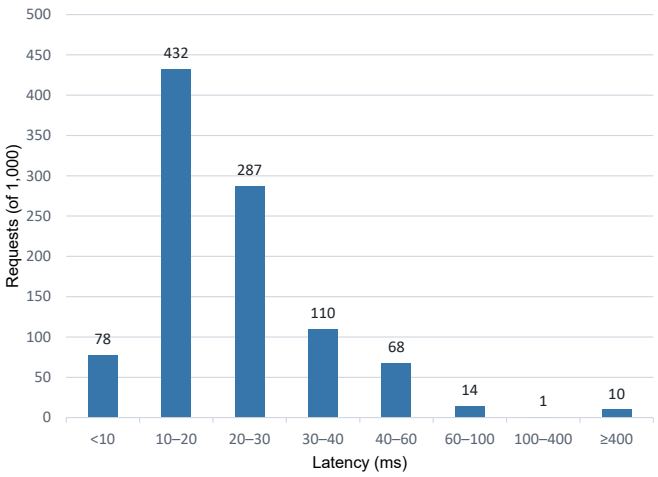
The goals conflict: batching work raises throughput but makes each item wait longer.

Latency: Orders of Magnitude

Data passing	Latency	Light in fibre travels	One at a time: max rate
Python function call (CPython 3.14)	≈ 20 ns	4 m	50,000,000 / s
Shared memory between cores	200 ns	40 m	5,000,000 / s
System V shared memory (IPC)	2 µs	400 m	500,000 / s
Low-latency network	8 µs	1.6 km	125,000 / s
Typical LAN round trip	30 µs	6 km	≈ 33,000 / s
Typical data-grid request	800 µs	160 km	1,250 / s
60 Hz power flicker	8 ms	1,600 km	120 / s
4G mobile request	55 ms	11,000 km	18 / s

After P. Lawrey (Chronicle Software); light covers ≈ 0.2 m per ns in fibre. Timings vary by machine.

Latency Is a Distribution



- Latencies are **skewed** with a long tail — not a normal distribution
- The mean mixes the typical case with rare outliers
- Report **percentiles**: p50 (median), p95, p99 and max
- p99 = 200 ms: 99% of requests take at most 200 ms

Key idea

Averages hide the tail, but users feel the tail.

Computing Latency Percentiles

percentiles.py

```python
import random
import statistics

random.seed(7)
# 1,000 simulated request latencies (ms): mostly fast, 1% very slow
latencies = [random.lognormvariate(3.0, 0.5) for _ in range(990)]
latencies += [random.uniform(400, 1500) for _ in range(10)]

cuts = statistics.quantiles(latencies, n=100)   # 99 cut points
print(f"mean  {statistics.fmean(latencies):6.1f} ms")
print(f"stdev {statistics.stdev(latencies):6.1f} ms")
for p in (50, 95, 99):
    print(f"p{p}   {cuts[p - 1]:6.1f} ms")
print(f"max   {max(latencies):6.1f} ms")
```

OUTPUT

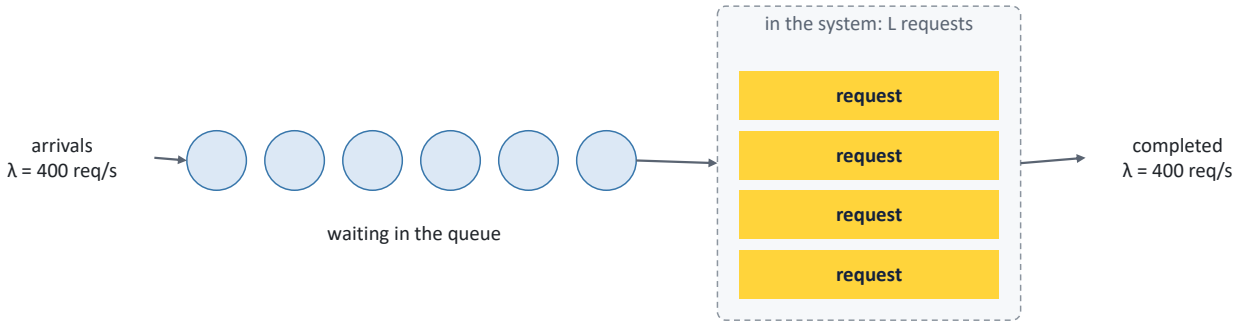
```
mean    29.9 ms
stdev   80.8 ms
p50     19.8 ms
p95     48.9 ms
p99    422.3 ms
max   1389.0 ms
```

Watch out

"Mean ± stdev" would suggest 29.9 ± 80.8 ms, i.e. negative latencies: standard deviation is meaningless for skewed data.

1 · APPLICATION PERFORMANCE

Little's Law



- **L = λ × W** for any stable system (Little, 1961)
- L: mean number of requests in the system (concurrency)
- λ: mean throughput; W: mean time in the system (latency)
- So mean latency W = L / λ = concurrency ÷ throughput

Worked example

400 req/s × 0.05 s = **20 requests in flight** on average: a worker pool smaller than 20 must queue requests.

QUIZ

Quick Check: Little's Law

An API serves 250 requests per second and each request spends 80 ms in the system on average. How many requests are in flight on average?

A 3

B 20

C 31

D 200

ANSWER

Quick Check: Little's Law

An API serves 250 requests per second and each request spends 80 ms in the system on average. How many requests are in flight on average?

A 3	B 20
C 31	D 200

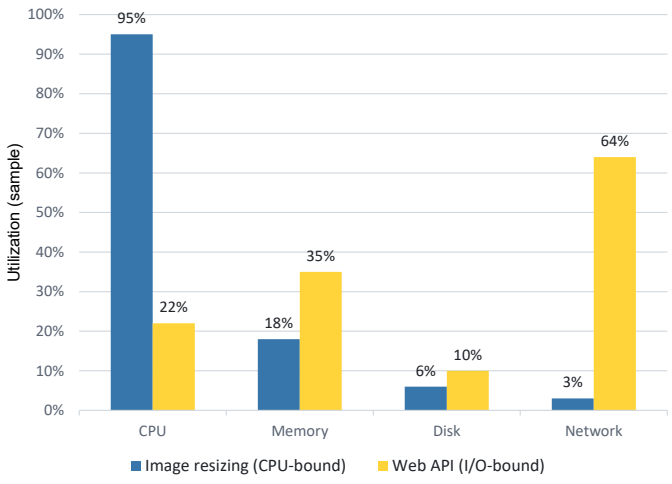
Why

Little's law: $L = \lambda \times W = 250 \text{ req/s} \times 0.08 \text{ s} = 20$ requests in the system on average.

1 · APPLICATION PERFORMANCE

Capacity and Utilization

- **Capacity**: units of work that can be in progress at the same time
- Quoted at a target, e.g. 800 req/s *at p99 ≤ 200 ms*
- **Utilization**: share of time a resource is busy
- Workloads saturate different resources



Efficiency

- **Efficiency** = throughput ÷ resources used
- More resources for the same throughput → less efficient
- Large systems: compare cost via **total cost of ownership** (TCO)

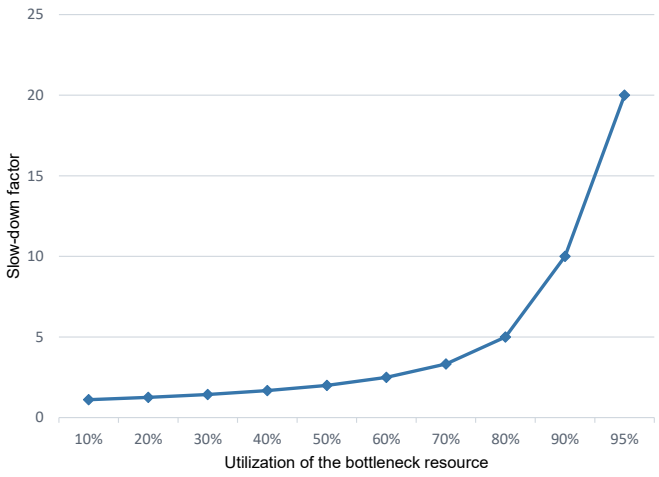
> ★ **Key idea**
> Same throughput at twice the cost = half as efficient.

	Solution A	Solution B
Throughput	2,000 req/s	2,000 req/s
Servers (8 cores each)	4	2
Monthly TCO	$4,000	$2,000
Req/s per core	62.5	125
Req/s per $1,000	500	1,000

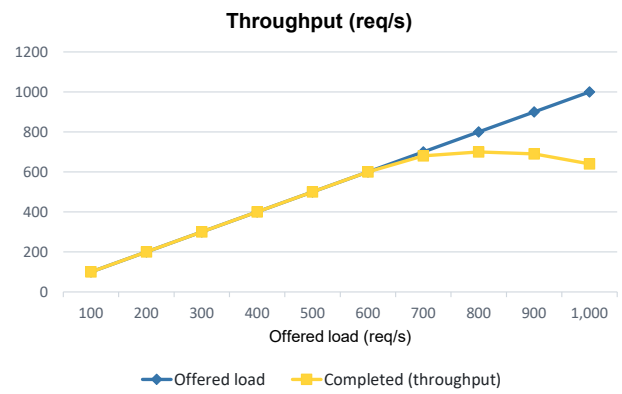
Degradation Under Load

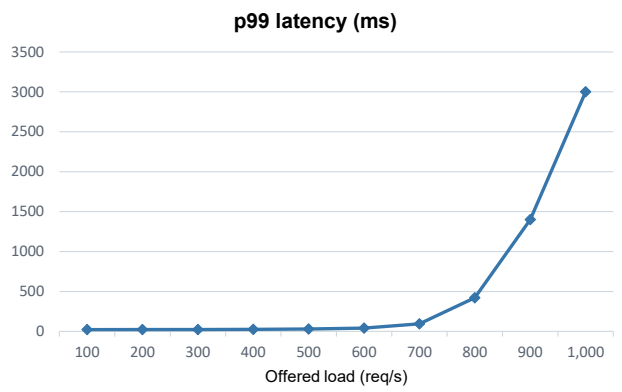
- More clients or faster arrivals change latency and throughput
- Under-utilized system: slack absorbs the extra load
- Near saturation, **queueing** makes latency explode
- Queueing rule of thumb: response time ∝ 1 / (1 − utilization)

> ! **Watch out**
> At 90% utilization, a request takes about 10× its bare service time.



Reading Graphs: Latency Under Load

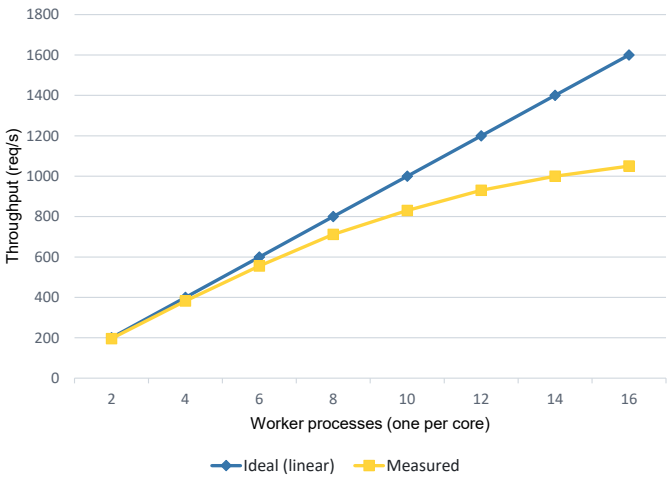




Key idea

Past capacity (~700 req/s) extra load only adds queueing: latency hits timeouts and throughput even *falls* as timed-out work is wasted.

Reading Graphs: Near-Linear Scaling



- Check the axes: units, linear or log scale, starting at zero?
- Compare with the **ideal** line, not with nothing
- Find where the curves diverge: the next bottleneck appears
- Look at variation too: one run is not a measurement

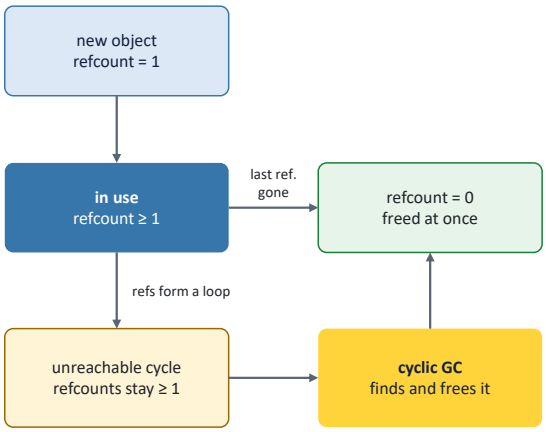
CPython Memory Management

Reference counting

- Every object counts the references to it
- Count reaches 0 → freed immediately
- Deterministic, spread over the whole run
- Cannot free reference cycles

Cyclic garbage collector

- Finds unreachable reference cycles
- Tracks only container objects
- Generational: young objects scanned most often
- Triggered by allocation counts



Key idea

Reference counting frees most memory at once, so the collector only handles cycles and its pauses are usually short — but not always.

Reference Cycles and the gc Module

a cycle survives del

```
>>> import gc
>>> class Node:
...     def __init__(self):
...         self.partner = None
...
>>> a, b = Node(), Node()
>>> a.partner, b.partner = b, a
>>> del a, b
>>> gc.collect() >= 2
True
```

After `del`, each `Node` is still referenced by the other: only `gc.collect()` frees them.

long-lived data at start-up

```
import gc

gc.disable()              # no GC runs while loading
catalog = load_catalog()  # many long-lived objects
gc.freeze()               # exclude them from future scans
gc.enable()
```

Tip

`gc.freeze()` moves all current objects to a permanent generation: later collections skip them, and forked worker processes keep sharing their memory pages.

Measuring GC Pauses

gc_pauses.py

```python
import gc
import time

pauses: list[float] = []
started = 0.0


def on_gc(phase: str, info: dict[str, int]) -> None:
    global started
    if phase == "start":
        started = time.perf_counter()
    else:
        pauses.append((time.perf_counter() - started) * 1000)


gc.callbacks.append(on_gc)
records = [{"id": i, "tags": [i]} for i in range(1_000_000)]
gc.callbacks.remove(on_gc)
pauses.sort()
print(f"GC runs: {len(pauses)}")
print(f"median:  {pauses[len(pauses) // 2]:.3f} ms")
print(f"max:     {pauses[-1]:.1f} ms")
```

SAMPLE OUTPUT — TIMINGS VARY BY MACHINE

```
GC runs: 999
median:  0.035 ms
max:     45.2 ms
```

Key idea

Typical collections take microseconds; occasional scans of the old generation over millions of objects take tens of milliseconds.

Pauses Beyond the Garbage Collector

I/O waits

- Network round trips
- Waiting for the database
- Disk reads and writes

Operating system

- Scheduling delays and interrupts
- Page faults and swapping
- Stalls of 5 ms or more are common

Contention

- Locks, pools and queues
- The GIL in multi-threaded, CPU-bound code

Runtime

- Cyclic GC runs
- First-call imports and warm-up
- Logging to slow sinks

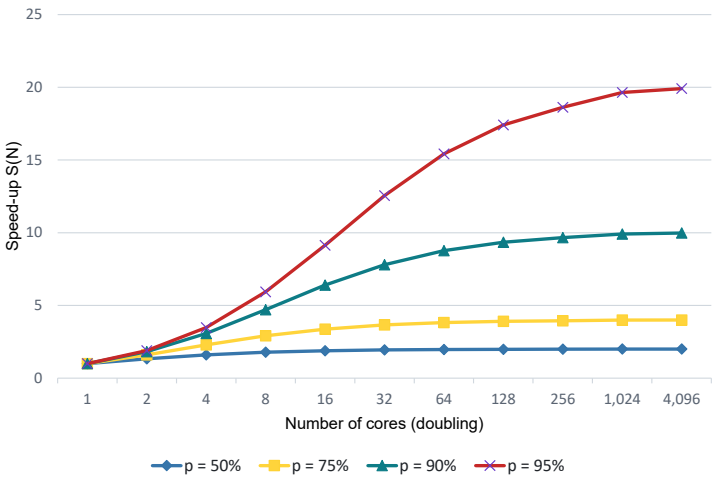
Key idea

Measure where a pause comes from before blaming the garbage collector.

Amdahl's Law

S(N) = 1 / ((1 − p) + p / N)

- p: fraction of the work that can run in parallel
- N: number of processors (cores)
- The serial part (1 − p) never gets faster
- Upper limit: S = 1 / (1 − p)
  - 95% parallel → at most 20×



Amdahl's Law: A Worked Example

amdahl.py

```python
def amdahl_speedup(parallel_fraction: float,
                   cores: int) -> float:
    """Upper bound on the speed-up of a partly parallel job."""
    serial_fraction = 1 - parallel_fraction
    return 1 / (serial_fraction + parallel_fraction / cores)


# 20-minute job: 4 min serial loading, 16 min parallel work
for cores in (1, 2, 4, 8, 16, 1_000):
    speedup = amdahl_speedup(0.8, cores)
    print(f"N = {cores:<5} S = {speedup:4.2f}x"
          f"  T = {20 / speedup:5.2f} min")
```

OUTPUT

```
N = 1     S = 1.00x  T = 20.00 min
N = 2     S = 1.67x  T = 12.00 min
N = 4     S = 2.50x  T =  8.00 min
N = 8     S = 3.33x  T =  6.00 min
N = 16    S = 4.00x  T =  5.00 min
N = 1000  S = 4.98x  T =  4.02 min
```

- p = 16 / 20 = 0.8
- 8 cores: 6 minutes, not 2.5
- Limit: 1 / 0.2 = 5×, never below 4 minutes
- Next step: shrink the serial part

Key idea

Halving the 4-minute serial load gains more than going from 16 to 1,000 cores.

TRY IT

Apply Amdahl's Law

A report generator runs for 100 s. Profiling shows that 90 s are spent in a loop whose iterations are independent; the remaining 10 s are serial.

1. What speed-up and run time do you get on 4 cores? On 16 cores?
2. What is the maximum possible speed-up?
3. Alternative: make the serial 10 s twice as fast and use 16 cores. Run time?

Formula

$S(N) = 1 / ((1 - p) + p / N)$

ANSWER

Apply Amdahl's Law

Scenario	Calculation	Speed-up	Run time
4 cores	1 / (0.1 + 0.9 / 4)	3.08×	32.5 s
16 cores	1 / (0.1 + 0.9 / 16)	6.40×	15.6 s
Infinite cores	1 / 0.1	10×	10 s
Serial 5 s + 16 cores	5 + 90 / 16	9.41×	10.6 s

Key idea

Speeding up the serial part got almost the full 10× limit with only 16 cores.

QUIZ

Quick Check: Latency Objectives

Your team writes a latency objective for a public web API. Which statistic should it use?

- **A** Mean latency over one day
- **B** Mean ± standard deviation
- **C** p99 latency over short windows (e.g. 5 minutes)
- **D** Minimum latency (best case)

ANSWER

Quick Check: Latency Objectives

Your team writes a latency objective for a public web API. Which statistic should it use?

- **A** Mean latency over one day
- **B** Mean ± standard deviation
- **C** p99 latency over short windows (e.g. 5 minutes)
- **D** Minimum latency (best case)

★ **Why**

Latency distributions are skewed with long tails. A high percentile over short windows describes what slow users experience and reveals problems quickly; means and standard deviations hide the tail.

02

Performance Methodology

Monitor, profile, tune — in that order

- Monitoring, profiling and tuning
- Performance in the development process
- Measurable non-functional requirements
- Top-down and bottom-up analysis

2 · PERFORMANCE METHODOLOGY

Monitoring, Profiling and Tuning

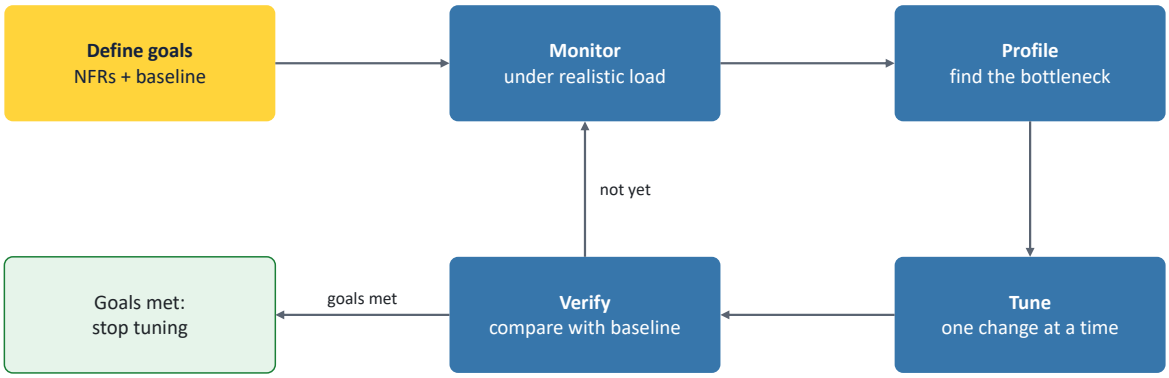
	Monitoring	Profiling	Tuning
What	Observe a running system and collect performance data	Measure where time and memory go inside a program	Change code, configuration or tunables to meet goals
Overhead	Non-intrusive: negligible	More intrusive: instrumentation or sampling	Changes the system itself
Focus	Broad: whole system, over time	Narrow: one program, one scenario	The bottleneck that profiling found
Where	Production, test and development	Development, test, qualification	Anywhere — verified by measurement
Tools	`top`, `vmstat`, Prometheus, Grafana	`cProfile`, `py-spy`, `tracemalloc`	Code, algorithms, OS and runtime settings

Key idea

Monitor to find the symptom, profile to find the cause, tune to remove it — then measure again.

2 · PERFORMANCE METHODOLOGY

The Tuning Loop



Tip

Change one thing at a time; otherwise you cannot tell which change helped.

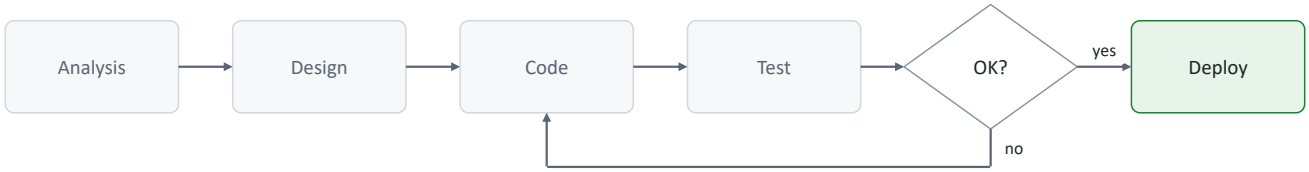
Watch out

Stop when the requirements are met: further tuning costs readability.

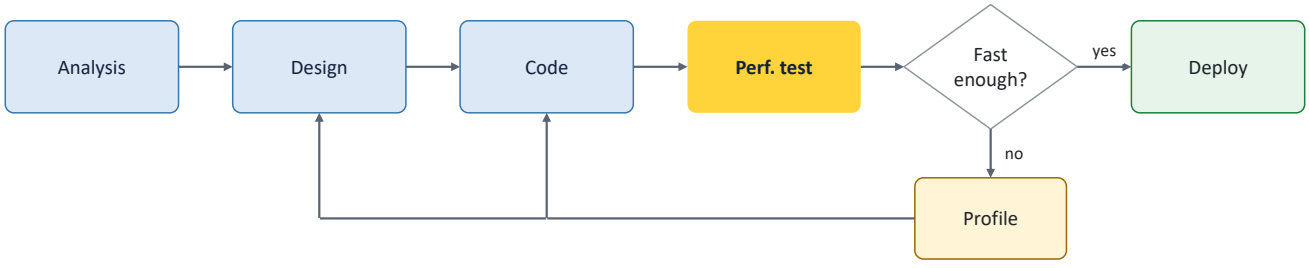
2 · PERFORMANCE METHODOLOGY

Performance in the Development Process

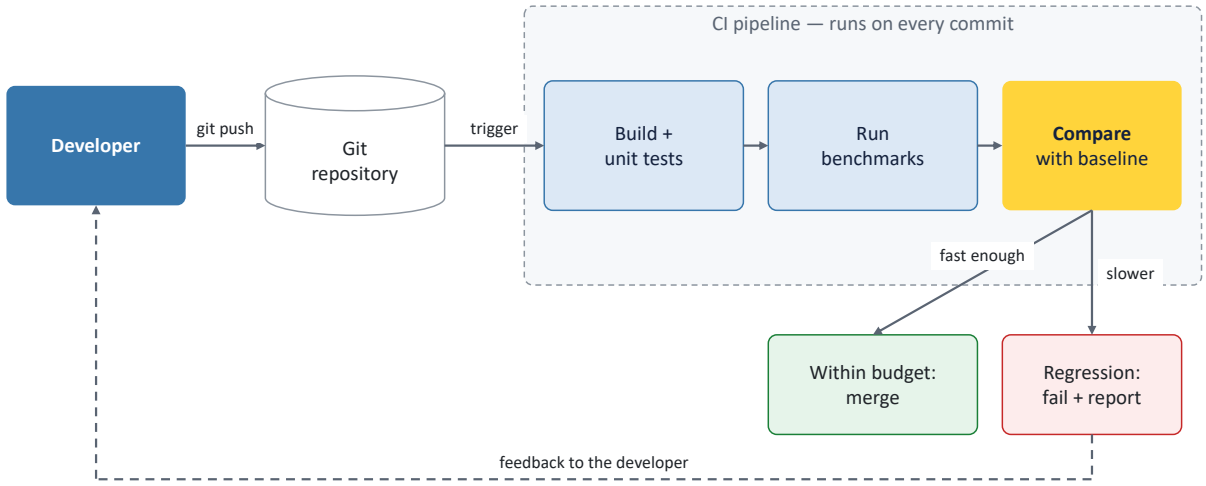
Traditional: performance is checked at the end, if at all



Performance-aware (Wilson & Kesselman): performance tests in every iteration



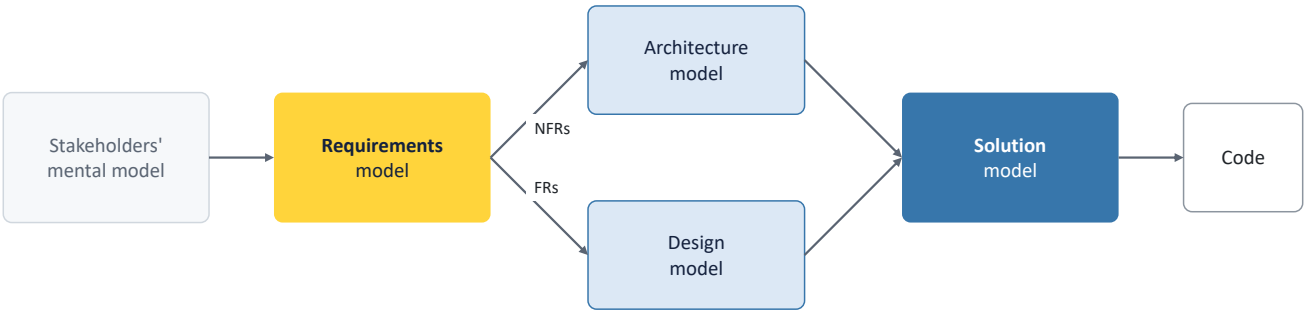
Continuous Performance Validation



Track performance at every commit: `pytest-benchmark` (section 5), `asv`, or CPython's own pyperformance suite on speed.python.org.

Systemic Qualities Drive Architecture

- **Systemic qualities**: non-functional requirements on the quality of service
  - Performance (responsiveness, latency), reliability (surviving failures), scalability (more load)
- They shape the **architecture** — cheap to address early, expensive to retrofit (Module 6)



2 · PERFORMANCE METHODOLOGY

Performance Requirements (NFRs)

Question to ask	Measurable answer (example)
Expected throughput?	≥ 500 orders/s sustained for 1 hour
Latency from request to response?	p95 ≤ 150 ms, p99 ≤ 300 ms
Concurrent users or tasks?	10,000 sessions, 200 requests in flight
Throughput and latency at peak concurrency?	At 200 in flight: ≥ 450 req/s, p99 ≤ 500 ms
Maximum worst-case latency?	No request > 2 s (hard timeout)
Tolerated pauses (GC, I/O stalls)?	≤ 1 pause > 100 ms per hour
Resource budget?	≤ 2 GB RSS per worker, startup ≤ 3 s

Tip

A requirement you cannot measure cannot be tuned for.

DISCUSS

Make These Requirements Measurable

Rewrite each vague requirement so that it could be checked by an automated test:

1. "The search page should be fast."
2. "The system must handle lots of users."
3. "Monthly reports should not take forever."
4. "The app should not use too much memory."

Hint

Name the metric, the statistic, the threshold and the conditions.

ANSWER

Make These Requirements Measurable

Vague	Measurable (one possible version)
Search is fast	p95 ≤ 200 ms, p99 ≤ 500 ms at 300 searches/s
Lots of users	5,000 concurrent sessions with p99 ≤ 1 s, error rate < 0.1%
Reports finish	Monthly report for 10 M rows completes in ≤ 15 min
Not too much memory	Peak RSS ≤ 1.5 GB per worker for a 24 h soak test

Key idea

Each version fixes a metric, a statistic, a threshold and the load at which it must hold.

2 · PERFORMANCE METHODOLOGY

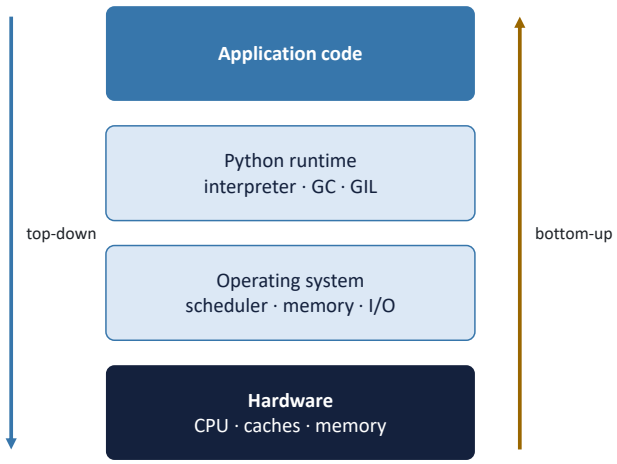
Top-Down and Bottom-Up Analysis

Top-down

- Start at the application under load
- Drill down the stack towards the problem
- Most common: you can change the code

Bottom-up

- Start at the CPU: cache misses, branch misses
- Find inefficient instructions and idioms
- Specialists; comparing platforms



Top-Down: Triggers and First Step

Typical triggers

- A stakeholder observes a problem at a certain load
- Monitoring shows degradation after a change in configuration or typical load
- Requirements change and the current system cannot meet them

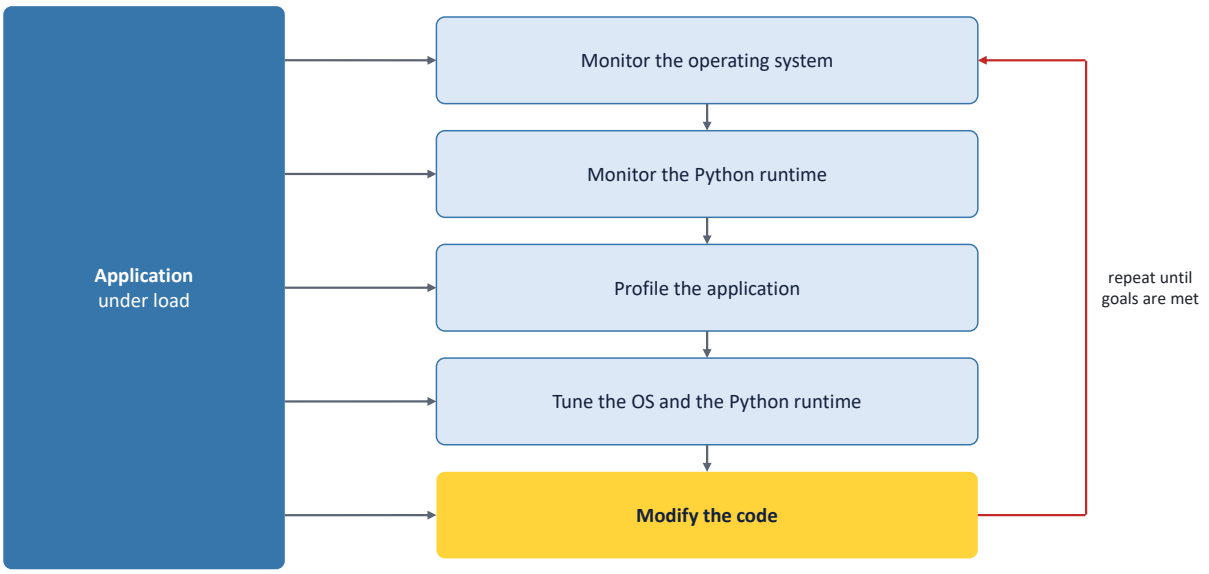
First step: monitor under the load of interest

Layer	What to observe
Operating system	CPU, memory, disk, network
Python runtime	GC activity, threads, GIL waits, imports
Application	Request timings, queue lengths, business metrics

Top-Down: Tools and Techniques

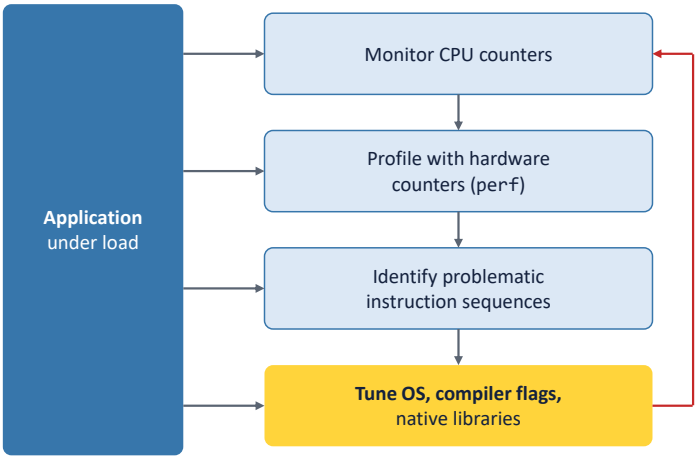
Technique	Answers the question	Typical tools
Metrics dashboards	How do latency, throughput, CPU and memory evolve?	Prometheus + Grafana, OpenTelemetry metrics
Code profiling	Which functions or lines consume the time?	`cProfile`, `py-spy`, `line_profiler` (section 5)
Log analysis	Which requests were slow, and when?	Structured logs, ELK, Grafana Loki
Database analysis	Which queries are slow, and why?	`EXPLAIN` plans, slow-query log, `explain()` (Module 7)
Distributed tracing	Where does a request spend time across services?	OpenTelemetry traces, Jaeger

The Top-Down Process



The Bottom-Up Process

- Used by performance specialists
- Goal: make an application perform as well on one platform as on another
- Platforms differ in CPU model, architecture (x86-64, ARM64) and core count
- Needs hardware performance counters: `perf` on Linux



2 · PERFORMANCE METHODOLOGY

Choosing and Evaluating a Platform

CPU architectures

- x86-64 (Intel, AMD)
- ARM64 (Apple silicon, AWS Graviton)
- RISC-V (emerging)

Specialized hardware

- GPU and TPU for tensor math
- ASIC and SoC designs
- ISA: the instruction-set contract

Evaluating performance

- Benchmark *your* workload
- `pyperformance` for Python itself
- Check wheels for the platform

TERMINAL — COMPARING TWO PLATFORMS WITH THE SAME BENCHMARK SUITE

```
$ python -m pip install pyperformance
$ python -m pyperformance run --fast -o py314-arm64.json
$ python -m pyperf compare_to py314-x86-64.json py314-arm64.json --table
```

★ **Key idea**

Python code is portable, but its speed depends on optimized native wheels (NumPy, cryptography, …) existing for the target architecture.

QUIZ

Quick Check: Choosing an Approach

Customers report that checkout became slow after last week's release. What is the best first step?

A. Count cache misses with `perf` (bottom-up)

B. Monitor the application under the load where checkout is slow (top-down)

C. Rewrite the checkout service in C

D. Tune the Linux scheduler

ANSWER

Quick Check: Choosing an Approach

Customers report that checkout became slow after last week's release. What is the best first step?

A. Count cache misses with `perf` (bottom-up)

B. Monitor the application under the load where checkout is slow (top-down)

C. Rewrite the checkout service in C

D. Tune the Linux scheduler

Why

The symptom is at application level and appeared after a change: start top-down by monitoring under the relevant load, then profile. Rewrites and OS tuning without data are guesses.

03

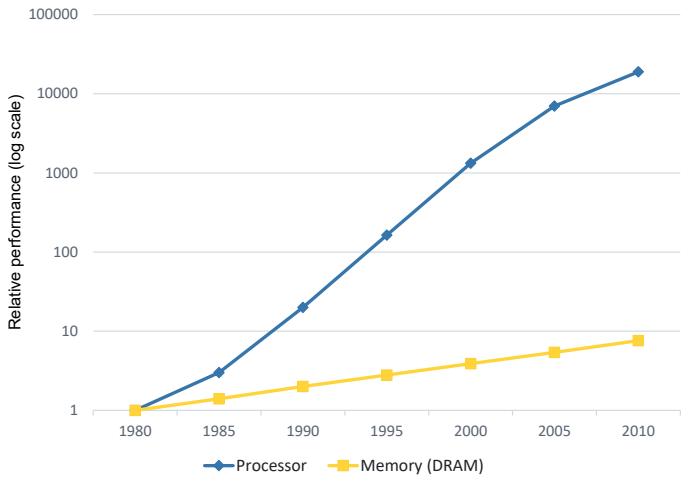
Hardware and OS Fundamentals

Why the machine behaves the way it does

- Memory, caches and the processor–memory gap
- Cache coherence (MESI) and the TLB
- Pipelines, branch prediction and memory models
- The OS scheduler and context switches

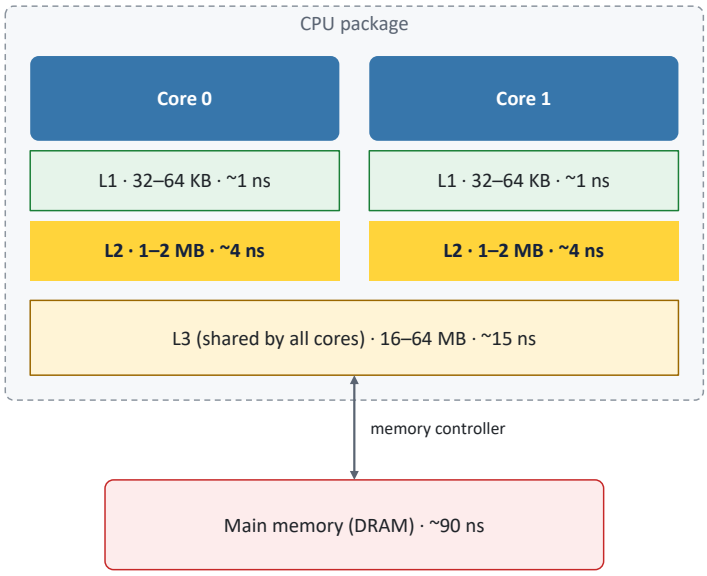
The Processor–Memory Gap

- **Moore's law**: transistor counts double roughly every two years
- Extra transistors first bought higher clock speeds
- Faster cores need a faster stream of data
- Memory latency improved far more slowly
- A core waiting for data idles, whatever its clock speed



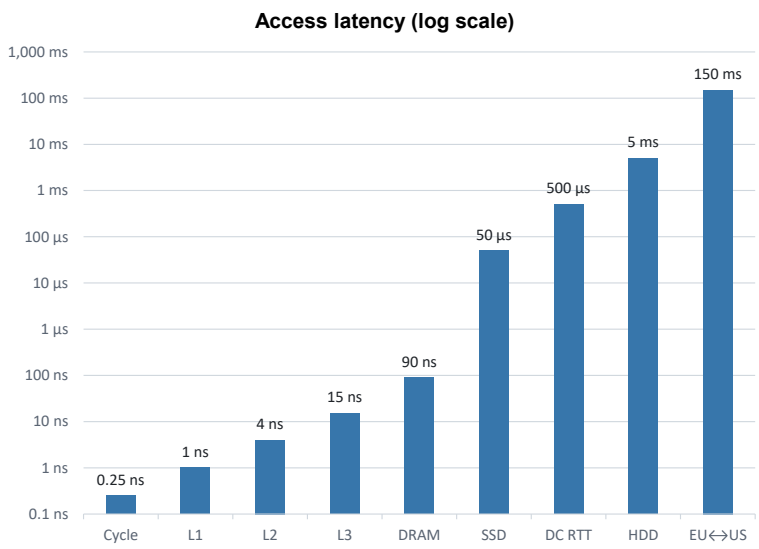
Approximate, after Hennessy & Patterson, Computer Architecture, 5th ed.

CPU Caches



- Small, fast memories on the CPU die
- Hold copies of recently used memory in 64-byte **cache lines**
- L1 and L2 per core, L3 shared by all cores
- **Locality** makes them work: reuse data soon, use its neighbours

Memory Hierarchy Latencies



If an L1 hit took 1 s…	
CPU cycle (4 GHz)	0.25 s
L1 cache hit	1 s
L2 cache hit	4 s
L3 cache hit	15 s
Main memory	1.5 min
NVMe SSD random read	14 hours
Data-centre round trip	6 days
HDD random read	2 months
Round trip Europe ↔ US West	4.8 years

Orders of magnitude for current x86-64 servers; after Hennessy & Patterson, J. Dean and C. Scott.

Locality in Practice: Rows vs Columns

locality.py

```python
import time

import numpy as np

grid = np.ones((4_000, 4_000))     # 128 MB, stored row by row

start = time.perf_counter()
row_total = sum(row.sum() for row in grid)       # contiguous
row_ms = (time.perf_counter() - start) * 1000

start = time.perf_counter()
col_total = sum(col.sum() for col in grid.T)     # 32 KB stride
col_ms = (time.perf_counter() - start) * 1000

print(f"row by row:       {row_ms:5.1f} ms")
print(f"column by column: {col_ms:5.1f} ms")
print(row_total == col_total)
```

SAMPLE OUTPUT — TIMINGS VARY BY MACHINE

```
row by row:        23.7 ms
column by column:  60.7 ms
True
```

- Same data, same arithmetic
- Rows: 8 values per 64-byte line
- Columns: a new line (and page) for every value

Key idea

Access order alone costs 2.5×.

Cache Coherence: The MESI Protocol

- Each core has private caches → copies of one memory line may diverge
- **Cache-coherence protocols** keep the copies consistent; the common one is **MESI**

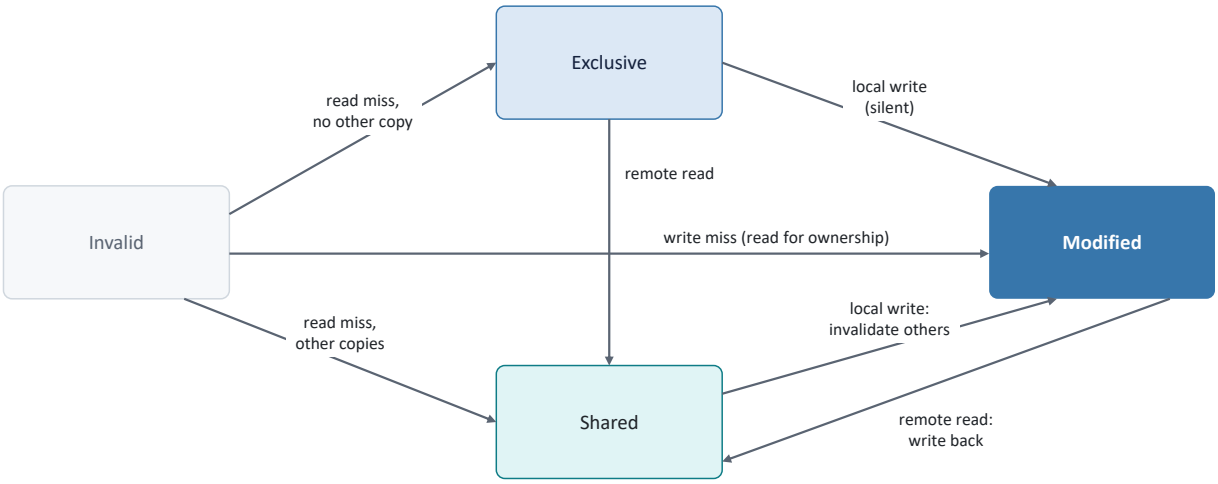
State	Meaning	Same as memory?	Other copies?
Modified	Written here, not yet written back	No	No
Exclusive	Only this cache holds it, clean	Yes	No
Shared	Clean; other caches may hold it	Yes	Maybe
Invalid	Unusable; dropped when convenient	—	—

	M	E	S	I
M	–	–	–	✓
E	–	–	–	✓
S	–	–	✓	✓
I	✓	✓	✓	✓

Allowed states of one line in two caches

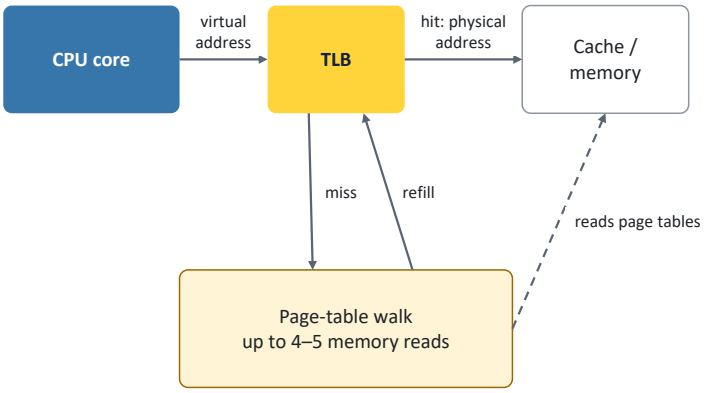
Key idea
Coherence is automatic but not free: writing to a shared line invalidates every other copy.

MESI State Transitions



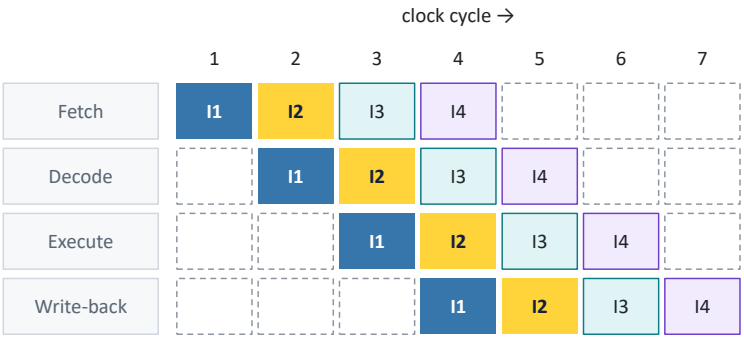
From any state a line becomes **Invalid** when another core writes it (a Modified line is written back first).

Translation Lookaside Buffer



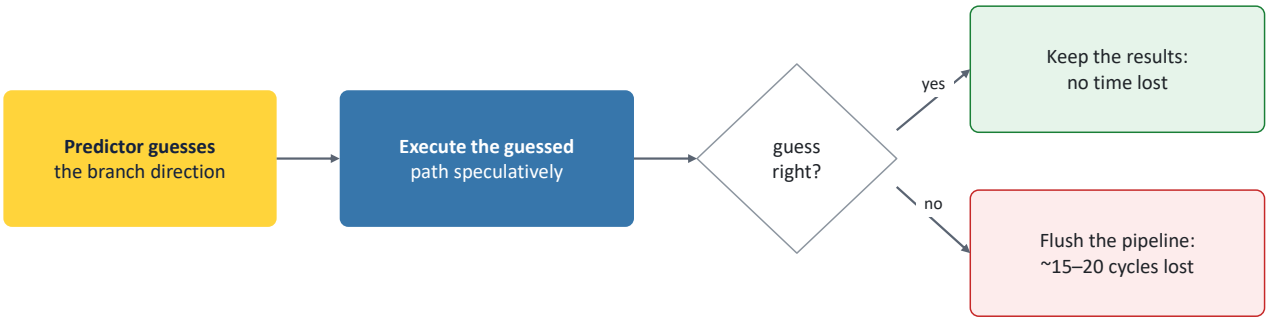
- Every memory access needs a virtual → physical translation
- Page tables live in memory: a full walk costs several accesses
- The **TLB** caches recent translations
- Large, scattered heaps cause TLB misses; **huge pages** (2 MB) help

Instruction Pipelines



- Executing one instruction takes several **stages**
- Different instructions occupy different stages at once
- From cycle 4, one instruction completes per cycle
- Modern cores: 14–20 stages, 4+ instructions per cycle

Branch Prediction and Speculative Execution



- The pipeline needs the next instruction before a branch is resolved
- Predictors learn from history: > 95% correct on typical code
- Random, data-dependent branches mispredict often

Note

CPython's interpreter branches on every bytecode; 3.11+ specialization keeps those branches predictable. Speculation also enabled Spectre and Meltdown (2018).

Hardware Memory Models

Compilers and CPUs may **reorder** independent operations: invisible to the thread itself, visible to others.

unsafe hand-off

```
# thread A (writer)
result = compute()
ready = True        # may become visible first!

# thread B (reader)
while not ready:    # busy-waits on a core
    pass
use(result)         # could see a stale result
```

Key idea

Python has no formal memory model: hand data over with `Event`, `Lock` or `Queue` (Module 5).

safe_handoff.py

```
import threading

ready = threading.Event()
results: list[int] = []


def reader() -> None:
    ready.wait()          # returns after set()
    print(results[0])


if __name__ == "__main__":
    consumer = threading.Thread(target=reader)
    consumer.start()
    results.append(sum(range(1_000)))
    ready.set()           # then publish
    consumer.join()
```

OUTPUT

```
499500
```

Memory Ordering by Architecture

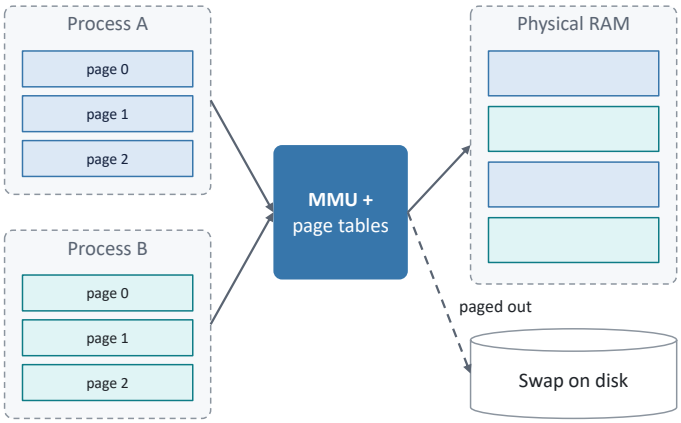
May an earlier access be reordered with a later one?	ARM (v7, v8)	POWER	RISC-V (RVWMO)	x86-64
Load → later load	Yes	Yes	Yes	No
Load → later store	Yes	Yes	Yes	No
Store → later store	Yes	Yes	Yes	No
Store → later load	Yes	Yes	Yes	Yes
Loads/stores around atomic operations	Yes	Yes	Yes	No

Watch out

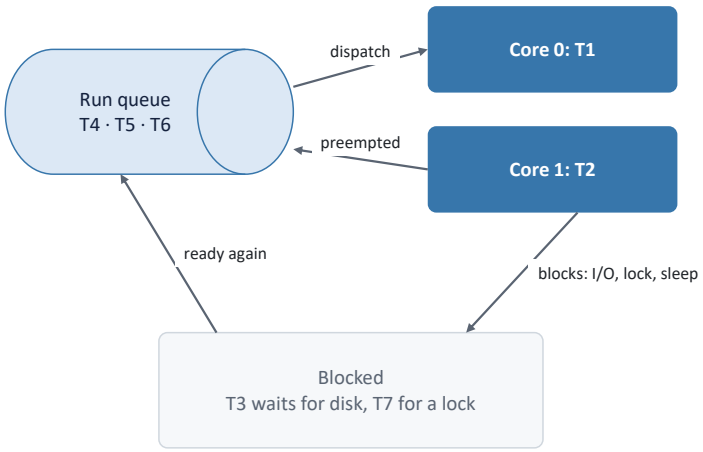
x86-64 is strongly ordered (TSO); ARM and RISC-V are weakly ordered. Code with unsynchronized hand-offs may work on a laptop and fail on an ARM server.

Operating Systems and Virtual Memory

- The OS arbitrates access to shared, finite resources
- All resources are finite; all processes are greedy
- The scarcest two: **memory** and **CPU time**
- The **MMU** and page tables give each process its own virtual address space



The Scheduler and the Run Queue



- More runnable threads than cores → they wait in the **run queue**
- The scheduler hands out time slices and responds to interrupts
- Linux: CFS, replaced by EEVDF in kernel 6.6
- Run queue longer than the core count = CPU saturation (`vmstat` column `r`)

Context Switches

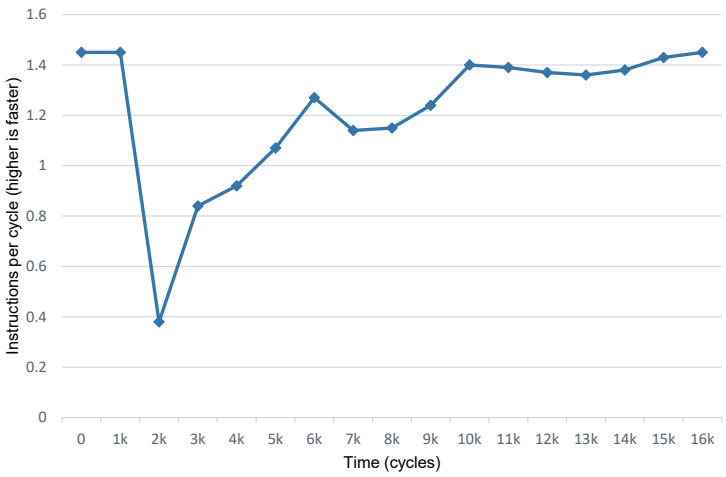


- The OS saves one thread's registers, stack pointer and program counter, then restores another's
- Direct cost: microseconds of kernel work
- Indirect cost: the new thread finds cold caches and TLB
- System calls switch user → kernel mode: cheaper, but also pollute the caches

Key idea

Thousands of switches per second are normal; hundreds of thousands per core point to contention or oversubscription.

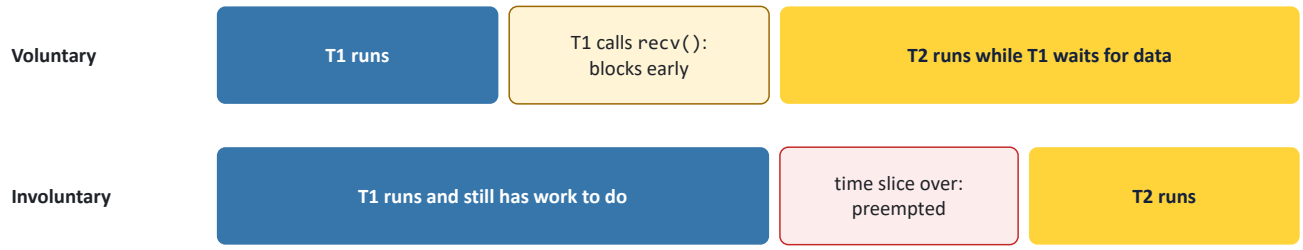
The Hidden Cost of System Calls



- The call itself is short
- Kernel code evicts user data from caches, TLB and predictors
- User code then runs slower for thousands of cycles
- Python: prefer buffered I/O and batching to many tiny calls

After Soares & Stumm, FlexSC (OSDI 2010): one `pwrite` call on an Intel Core i7.

Voluntary vs Involuntary Context Switches



	Voluntary (`cswch/s`)	Involuntary (`nvcswch/s`)
Cause	The thread blocks: I/O, lock, sleep, waiting for the GIL	Time slice expired or a higher-priority thread woke up
High count suggests	I/O-bound work, lock or GIL contention	More busy threads than cores (oversubscription)
Python example	Four CPU-bound threads taking turns on the GIL	`Pool(processes=32)` on an 8-core machine

QUIZ

Quick Check: Context Switches

A thread calls `sock.recv(4096)` and no data has arrived yet. What happens?

- **A** The thread spins on its core until data arrives
- **B** The thread blocks and the scheduler performs a voluntary context switch
- **C** The thread is preempted: an involuntary context switch
- **D** The CPU speculatively executes the rest of the function

ANSWER

Quick Check: Context Switches

A thread calls `sock.recv(4096)` and no data has arrived yet. What happens?

- **A** The thread spins on its core until data arrives
- **B** The thread blocks and the scheduler performs a voluntary context switch
- **C** The thread is preempted: an involuntary context switch
- **D** The CPU speculatively executes the rest of the function

★ **Why**

A blocking `recv()` puts the thread to sleep in the kernel until data arrives; it gives up the core before its time slice ends, which is counted as a voluntary switch.

TRY IT

Estimate the Cost of Cache Misses

A loop reads 10 million 8-byte values once. Estimate the time spent waiting for memory:

1. Every read hits L1 (≈ 1 ns)
2. Every read misses all caches and goes to DRAM (≈ 90 ns)
3. Reads are sequential: one DRAM miss per 64-byte line, the other reads hit L1

✓ **Bonus**

Which case resembles iterating over a Python list of objects scattered on the heap?

ANSWER

Estimate the Cost of Cache Misses

Case	Calculation	Time
1 · all L1 hits	10,000,000 × 1 ns	10 ms
2 · all DRAM misses	10,000,000 × 90 ns	900 ms
3 · sequential	1,250,000 × 90 ns + 8,750,000 × 1 ns	≈ 121 ms

★ **Key idea**

Access pattern alone changes the time by 7–90×; hardware prefetching makes case 3 even faster. Scattered objects behave like case 2.

04

Monitoring CPU, Memory, Disk and Network

Reading the symptoms with Linux and Windows tools

- Why monitor: symptoms, the USE method and a system model
- CPU: `vmstat`, `mpstat`, `top`, `pidstat`, `perf`
- Memory, disk and network: paging, `iostat`, `iotop`, `ss`, `sar`
- Monitoring from Python; Windows: Task Manager, perfmon, `typeperf`

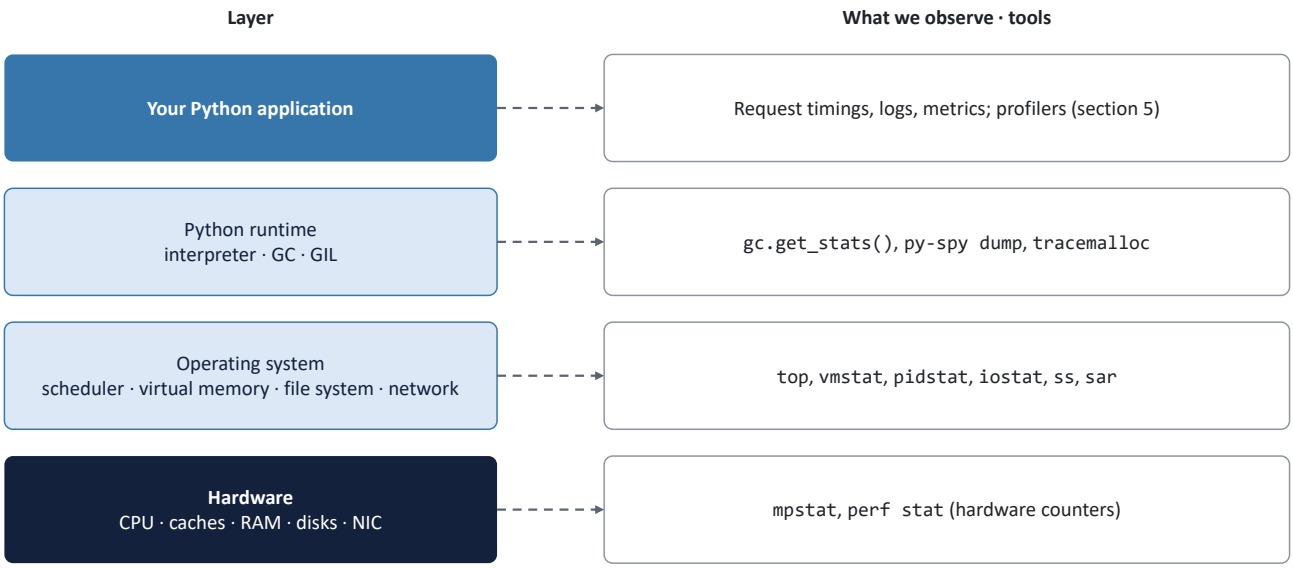
Why Monitor?

- Get a sense of what the problem is
- Identify the symptoms of poor performance
- From the symptoms, diagnose the cause and pick the next tool
- Compare with a **baseline** of normal behaviour

The USE method (B. Gregg): for every resource check

Resource	Utilization	Saturation	Errors
CPU	`us` + `sy` (`mpstat`)	run queue `r` > cores	—
Memory	used vs available (`free`)	swapping (`si`, `so`)	OOM kills (`dmesg`)
Disk	`%util` (`iostat -x`)	queue `aqu-sz`, `await`	I/O errors (`dmesg`)
Network	throughput vs link speed	drops, retransmits	`rxerr`, `txerr`

A Simple System Model



Monitoring CPU Usage

Metric	Meaning	Look closer when…
`us` — **user time**	Running application code (the Python interpreter)	Below ~100% while a CPU-bound job is slow
`sy` — **system time**	Kernel work: system calls, page faults, locking	High: too many syscalls or contention
`id` — **idle time**	Nothing to run	High under load: the application cannot scale
`wa` — **iowait**	Idle while waiting for disk I/O	High: disk-bound (next: `iostat`)
`st` — **steal time**	Virtual machine waiting for the hypervisor	High: noisy neighbours
VCX / ICX	Voluntary / involuntary context switches	Many VCX: waiting, locks, GIL; many ICX: too many threads

Key idea

High system time together with idle CPU often means contention: threads wait on a shared resource instead of working.

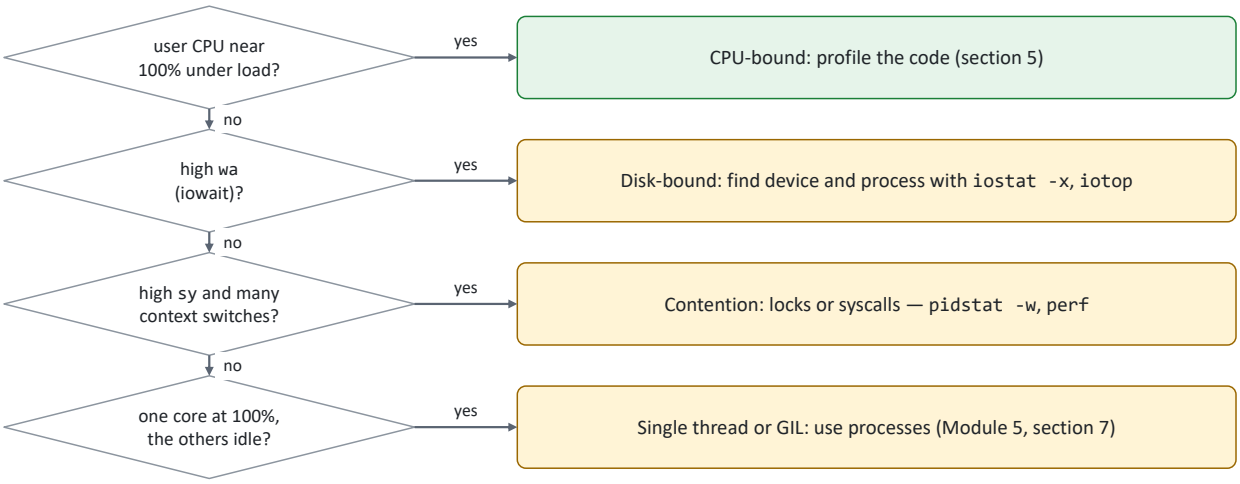
CPU Monitoring Tools

Tool	Scope	Typical question
`top` / `htop`	Per process, interactive	Who is using the CPU right now?
`vmstat 5`	Whole system	Saturated? `r`, `us`/`sy`/`id`/`wa`, context switches
`mpstat -P ALL 5`	Per CPU	Is one core busy while the others idle?
`pidstat -u -w 5`	Per process or thread	CPU and context switches over time
`sar -u`	History (sysstat collector)	What happened at 3 a.m.?
`perf stat` / `perf top`	Hardware counters, hot code	IPC, cache and branch misses
Task Manager, `resmon`, `perfmon`	Windows	The same questions on Windows

Note

Replacing the Solaris tools of the original: `prstat` → `top`, `pidstat`; DTrace scripts → `perf` and eBPF tools (`bpftrace`, BCC).

Is the CPU Busy Enough?



None of these? The program is waiting for the network or other services: check `ss` and tracing.

vmstat: A System-Wide Snapshot

TERMINAL — SAMPLE OUTPUT (8-CPU SERVER RUNNING A CPU-BOUND BATCH JOB)

```
$ vmstat -S M 5 4
procs -----------memory---------- ---swap-- -----io---- -system-- ------cpu-----
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st
 1  0      0   3139    148   4521    0    0    21    35  410  780  6  2 92  0  0
 9  0      0   2745    148   4521    0    0     0    18 8120 3410 96  3  1  0  0
10  0      0   2741    148   4521    0    0     0    22 8217 3388 97  3  0  0  0
 8  0      0   2739    148   4521    0    0     0    41 8094 3502 96  4  0  0  0
```

- First line: averages since boot — skip it
- `r` = 8–10 runnable on 8 CPUs: the CPU is saturated
- `us` 96–97%: the time goes to user code
- `si`/`so` = 0: no swapping
- `wa` = 0: disks are not the problem
- Next step: profile the Python code

Reading vmstat Columns

Group	Columns	Meaning
procs	`r` `b`	Runnable (running or waiting for a CPU); blocked in uninterruptible I/O
memory	`swpd` `free` `buff` `cache`	Swap used, free memory, buffers, page cache (KiB, or MiB with `-S M`)
swap	`si` `so`	Memory swapped in from / out to disk per second (KiB)
io	`bi` `bo`	Blocks read from / written to block devices per second (1 KiB blocks)
system	`in` `cs`	Interrupts and context switches per second
cpu	`us` `sy` `id` `wa` `st`	% of CPU time: user, system, idle, I/O wait, stolen

Key idea

`us` + `sy` + `id` + `wa` + `st` ≈ 100, so `us` + `sy` = 100 − `id` only when `wa` and `st` are zero.

DISCUSS

Interpret a vmstat Snapshot

TERMINAL — SAMPLE OUTPUT (8-CPU SERVER DURING A NIGHTLY EXPORT)

```
$ vmstat -S M 5 3
procs -----------memory---------- ---swap-- -----io---- -system-- ------cpu-----
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st
 1  4      0   1874     96  11210    0    0    12 98340 3120 5410  7  6 26 61  0
 0  5      0   1866     96  11226    0    0     8 97712 3055 5302  6  6 27 61  0
 1  4      0   1859     96  11241    0    0    10 99105 3190 5488  7  7 25 61  0
```

1. Is the CPU the bottleneck?
2. What are the processes in column `b` doing?
3. Is memory a problem?
4. Which tool would you run next, and why?

ANSWER

Interpret a vmstat Snapshot

Question	Evidence	Conclusion
CPU?	`us` + `sy` ≈ 13%, `r` ≤ 1	No — the CPU mostly waits
Column `b`?	4–5 blocked, `wa` 61%	Waiting for disk I/O
Memory?	`si` = `so` = 0, `cache` growing	No swapping; files are being written
Next tool?	`bo` ≈ 96 MB/s all the time	`iostat -x` (which device, how busy), `iotop` (which process)

Key idea

Disk-bound: a faster disk, fewer or larger writes, or compression help — faster Python code does not.

mpstat: Per-CPU Utilization

TERMINAL — SAMPLE OUTPUT (A PYTHON JOB WITH 8 CPU-BOUND THREADS)

```
$ mpstat -P ALL 5 1
Linux 6.8.0-45-generic (lab-server)   09/27/2026   _x86_64_   (8 CPU)

10:42:15     CPU    %usr   %nice    %sys %iowait    %irq   %soft  %steal  %guest  %gnice   %idle
10:42:20     all   12.60    0.00    0.45    0.00    0.00    0.03    0.00    0.00    0.00   86.92
10:42:20       0    0.20    0.00    0.60    0.00    0.00    0.20    0.00    0.00    0.00   99.00
10:42:20       1    0.20    0.00    0.40    0.00    0.00    0.00    0.00    0.00    0.00   99.40
10:42:20       2    0.00    0.00    0.60    0.00    0.00    0.00    0.00    0.00    0.00   99.40
10:42:20       3   99.80    0.00    0.20    0.00    0.00    0.00    0.00    0.00    0.00    0.00
10:42:20       4    0.20    0.00    0.40    0.00    0.00    0.00    0.00    0.00    0.00   99.40
10:42:20       5    0.20    0.00    0.60    0.00    0.00    0.00    0.00    0.00    0.00   99.20
10:42:20       6    0.00    0.00    0.40    0.00    0.00    0.00    0.00    0.00    0.00   99.60
10:42:20       7    0.20    0.00    0.40    0.00    0.00    0.00    0.00    0.00    0.00   99.40
```

Key idea

12.6% on average hides one saturated core: the threads take turns on the GIL, so only one runs Python code at a time.

top: Per-Process View

TERMINAL — SAMPLE OUTPUT

```
$ top
top - 10:42:31 up 12 days,  3:07,  2 users,  load average: 1.04, 0.88, 0.61
Tasks: 287 total,   2 running, 285 sleeping,   0 stopped,   0 zombie
%Cpu(s): 12.6 us,  0.4 sy,  0.0 ni, 86.9 id,  0.0 wa,  0.0 hi,  0.1 si,  0.0 st
MiB Mem :  15890.4 total,   2741.6 free,   8019.5 used,   5129.3 buff/cache
MiB Swap:   2048.0 total,   2048.0 free,      0.0 used.   7870.9 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
  48213 ada       20   0 1843520 912344  14208 R  99.7   5.6   4:12.83 python3
   1187 root      20   0 2914640  38212  26980 S   1.0   0.2  12:40.05 containerd
   2210 ada       20   0  756320  81244  45120 S   0.7   0.5   0:31.12 code
```

- `python3` at 99.7%: exactly one core busy
- `RES` 891 MiB: physical memory in use
- Load average ≈ 1: one runnable thread on average
- Keys: `1` per-CPU lines, `H` threads, `P` / `M` sort by CPU / memory
- `top -H -p 48213`: the threads of one process
- `htop`: colour bars, tree view, mouse support

pidstat: Threads and Context Switches

TERMINAL — SAMPLE OUTPUT (4 WORKER THREADS, STANDARD CPYTHON BUILD)

```
$ pidstat -w -t -p 48213 5 1
Linux 6.8.0-45-generic (lab-server)   09/27/2026   _x86_64_   (8 CPU)

10:45:02      UID      TGID       TID   cswch/s nvcswch/s  Command
10:45:07     1000     48213         -      0.20      0.00  python3
10:45:07     1000         -     48213      0.20      0.00  |__python3
10:45:07     1000         -     48220     98.40      2.60  |__python3
10:45:07     1000         -     48221    101.20      1.80  |__python3
10:45:07     1000         -     48222     99.60      2.20  |__python3
10:45:07     1000         -     48223    100.40      2.00  |__python3
```

- `cswch/s`: voluntary switches — the thread blocked
- `nvcswch/s`: involuntary — the thread was preempted
- `-t` lists every thread (TID) of the process

Key idea

~100 voluntary switches per second per thread, with little real waiting: the threads keep handing the GIL to each other.

perf: Hardware Counters

TERMINAL — SAMPLE OUTPUT

```
$ perf stat -e cycles,instructions,cache-misses,branch-misses python3 simulate.py

 Performance counter stats for 'python3 simulate.py':

    12,843,562,118      cycles
    31,207,934,605      instructions              #    2.43  insn per cycle
        18,452,337      cache-misses
        41,988,120      branch-misses

       3.214585931 seconds time elapsed

       3.182104000 seconds user
       0.031998000 seconds sys
```

- IPC = instructions ÷ cycles: > 2 good, < 1 mostly stalled
- Cache and branch misses: clues for the bottom-up approach
- `perf top`: hottest functions, system-wide
- Python 3.12+: `python -X perf` shows Python function names in `perf`

QUIZ

Quick Check: One Busy Core

Your 8-thread, CPU-bound Python program (standard CPython build) is slow. `mpstat` shows one CPU near 100% user time and seven almost idle. Most likely cause?

A. The disk is too slow

B. The GIL lets only one thread execute Python bytecode at a time

C. The Linux scheduler cannot handle 8 threads

D. The program is swapping

ANSWER

Quick Check: One Busy Core

Your 8-thread, CPU-bound Python program (standard CPython build) is slow. `mpstat` shows one CPU near 100% user time and seven almost idle. Most likely cause?

A. The disk is too slow

B. The GIL lets only one thread execute Python bytecode at a time

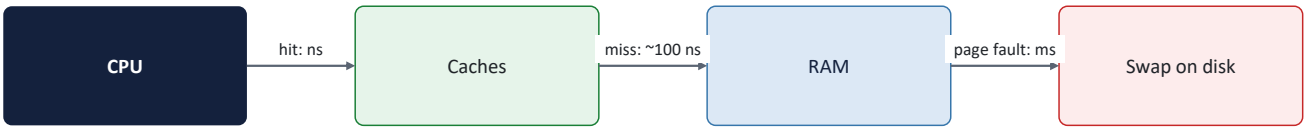
C. The Linux scheduler cannot handle 8 threads

D. The program is swapping

★ **Why**

CPU-bound threads in the default build take turns on the GIL, so together they use about one core. Use processes (`ProcessPoolExecutor`), native code that releases the GIL, or the optional free-threaded build.

Monitoring Memory: Paging and Swapping



Observe

- `si`, `so` in `vmstat`: swapped in / out per second
- Major page faults: `pidstat -r` (`majflt/s`)
- Page scanning: `sar -B` (`pgscank/s`)

Fix

- Add physical memory
- Run fewer processes on the machine
- Shrink the footprint (sections 6–7)

> **Watch out**
> Swapping hurts Python badly: GC scans and reference-count updates touch objects all over the heap, so swapped-out pages keep coming back.

Memory: A Swapping Example

TERMINAL — SAMPLE OUTPUT (A WORKER PROCESS LEAKS MEMORY)

```
$ vmstat -S M 5
procs -----------memory---------- ---swap-- -----io---- -system-- ------cpu-----
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st
 1  0     12    793     19    392    0    0     8    14  502  910  9  2 89  0  0
 2  1     50     96      2    113   64 7821    92  7840 2140 3350 21 14 41 24  0
 1  4     98     60      1     82 4436 9912  4630  9960 3930 5120  8 12 22 58  0
 1  3    143     57      1     78 9110 9240  9322  9281 3310 4870  6 10 28 56  0
$ free -h
               total        used        free      shared  buff/cache   available
Mem:            15Gi        15Gi        57Mi        12Mi        79Mi        88Mi
Swap:          2.0Gi       143Mi       1.9Gi
```

- `free` drops to ~60 MB and the page cache is squeezed out (`cache` 392 → 78)
- `so`, then `si` rise: pages go out to swap and must come back (thrashing)
- `b` and `wa` climb; next: which process grows? `pidstat -r 5` or `top` sorted by `RES`

Monitoring Disk I/O

Metric	Meaning	Linux tool
IOPS	Reads and writes per second (`r/s`, `w/s`, `tps`)	`iostat -x`, `sar -d`
Throughput	kB read and written per second	`iostat`, `sar -d`
Latency	Average time per request incl. queueing (`await`, ms)	`iostat -x`, `sar -d`
Queue size	Requests waiting or in service (`aqu-sz`)	`iostat -x`, `sar -d`
Utilization	Share of time the device was busy (`%util`)	`iostat -x`, `sar -d`
Per process	Who reads and writes how much	`iotop`, `pidstat -d`

Note

The page cache (`buff/cache`) absorbs reads and delays writes; `fsync()` forces data to disk. For SSDs `%util` near 100% is not yet saturation — watch `await`.

Disk Tools in Action: sar and iotop

TERMINAL — SAMPLE OUTPUT

```
$ sar -d -p 5 1
10:51:03          DEV       tps     rkB/s     wkB/s     dkB/s   areq-sz    aqu-sz     await     %util
10:51:08      nvme0n1   2841.20      6.40 363667.20      0.00    128.00     11.00      3.87     98.60
10:51:08          sda      0.40      0.00      3.20      0.00      8.00      0.00      0.50      0.04
```

TERMINAL — SAMPLE OUTPUT

```
$ sudo iotop -o -b -n 1
Total DISK READ:         0.00 B/s | Total DISK WRITE:       352.18 M/s
Current DISK READ:       0.00 B/s | Current DISK WRITE:     355.02 M/s
    TID  PRIO  USER     DISK READ  DISK WRITE  SWAPIN     IO>    COMMAND
  51877 be/4 ada         0.00 B/s  351.95 M/s  0.00 %  92.41 % python3 export.py
```

Key idea

Little's law again: `aqu-sz` ≈ `tps` × `await` = 2,841/s × 3.87 ms ≈ 11 requests queued at the device.

Monitoring Network I/O

What to watch	Why	Linux tools
Throughput (kB/s, packets/s)	Is the link saturated?	`sar -n DEV`, `iftop`
Connections and their states	Leaks, pool sizing, `TIME_WAIT` build-up	`ss -s`, `ss -tanp`
Retransmits, drops, errors	Packet loss or congestion	`nstat`, `sar -n ETCP,EDEV`
Traffic per process	Who sends and receives what?	`nethogs`
Round-trip time per connection	Slow peer or network path	`ss -ti`, `ping`

Note

Original tools → Linux: `netstat` → `ss`; `nicstat` → `sar -n DEV`; DTrace `tcptop` → `nethogs` or BCC `tcptop`. Windows: Resource Monitor's Network tab, perfmon counters.

Network Tools in Action: sar and nethogs

TERMINAL — SAMPLE OUTPUT (1 GBIT/S INTERFACE)

```
$ sar -n DEV 1 1
10:53:11        IFACE   rxpck/s   txpck/s    rxkB/s    txkB/s   rxcmp/s   txcmp/s  rxmcst/s   %ifutil
10:53:12           lo      2.00      2.00      0.12      0.12      0.00      0.00      0.00      0.00
10:53:12         eth0   8123.00  81542.00    531.87 117340.62      0.00      0.00      0.00     96.13
```

TERMINAL — SAMPLE OUTPUT

```
$ sudo nethogs eth0
    PID USER     PROGRAM                          DEV         SENT      RECEIVED
  48213 ada      python3 upload_backup.py         eth0   116842.305     512.411 KB/sec
   1190 root     sshd: ada@pts/0                  eth0        1.204       0.318 KB/sec
```

Key idea

≈ 117,340 kB/s ÷ 81,542 packets/s ≈ 1.44 kB per packet — close to the 1,500-byte MTU: a bulk upload. Many tiny packets would point to chatty request/response traffic.

Monitoring Processes

Data of interest	Linux tools
Footprint (resident memory)	`ps -o pid,rss,cmd`, `top` (`RES`), `pidstat -r`
Threads and thread states	`top -H`, `ps -eLf`, `/proc/PID/status`
CPU usage	`pidstat -u`, `top`
Runtime stack (what is it doing?)	`py-spy dump --pid PID` (section 5)
Context switches	`pidstat -w`, `/proc/PID/status`
Lock contention	High `cswch/s` + `sy`; `perf lock`, BCC `offcputime`

TERMINAL — SAMPLE OUTPUT

```
$ grep -E 'VmRSS|Threads|ctxt' /proc/48213/status
VmRSS:       912344 kB
Threads:     5
voluntary_ctxt_switches:     1204
nonvoluntary_ctxt_switches:  311
```


Monitoring the Kernel

TERMINAL — SAMPLE OUTPUT (8-CPU SERVER, 64 BUSY THREADS)

```
$ vmstat -S M 5 3
procs -----------memory---------- ---swap-- -----io---- -system-- ------cpu-----
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st
14  0      0   3047    199   5108    0    0     0    12 48210 412330 38 49 13  0  0
16  0      0   3045    199   5108    0    0     0     9 47895 405918 37 50 13  0  0
```

Kernel signal	Linux tool
`sy`, `in`, `cs`, run queue `r`	`vmstat 5`
Interrupts per CPU	`mpstat -I SUM -P ALL 5`
System calls by type	`perf trace -s`, `strace -c`
CPU migrations, run-queue latency	`perf stat`, BCC `runqlat`

Key idea

Half of all CPU time in the kernel and 400,000 switches per second: threads fight over a lock or make tiny system calls.

DISCUSS

Pick the Right Tool

#	Symptom
1	A web API is slow; overall CPU is only 15% on an 8-core server
2	A nightly export takes 3 hours and the server feels sluggish
3	A container is killed with exit code 137 (out of memory)
4	The same service is slower on a new cloud VM
5	Clients get connection errors; thousands of sockets in `TIME_WAIT`
6	The process is CPU-bound — which function is hot?

For each symptom: which tool do you run first, and what do you look for?

ANSWER

Pick the Right Tool

#	First tool(s)	Look for
1	`mpstat -P ALL`, `pidstat -u -t`	One core at 100% (GIL) — or idle CPUs waiting on I/O
2	`vmstat`, `iostat -x`, `iotop`	`wa`, `b`, device `%util` and `await`, the writer
3	`free`, `vmstat`, `pidstat -r`	Growing RSS, swapping; then `tracemalloc`
4	`vmstat` / `mpstat` (`st`), `perf stat`	Steal time, lower IPC, fewer cores
5	`ss -s`, `ss -tan state time-wait`	A new connection per request: add pooling
6	`py-spy`, `cProfile` (section 5)	Functions with the most cumulative time

Monitoring from Python with psutil

self_monitor.py

```python
import psutil  # third-party: pip install psutil

proc = psutil.Process()              # the current process
print(f"system CPU: {psutil.cpu_percent(interval=1):.1f} %")
print(f"RSS:        {proc.memory_info().rss / 2**20:.1f} MiB")
print(f"threads:    {proc.num_threads()}")
ctx = proc.num_ctx_switches()
print(f"switches:   {ctx.voluntary} voluntary, "
      f"{ctx.involuntary} involuntary")
net = psutil.net_io_counters()
print(f"network:    {net.bytes_sent:,} B sent")
```

SAMPLE OUTPUT

```
system CPU: 13.2 %
RSS:        27.4 MiB
threads:    1
switches:   41 voluntary, 3 involuntary
network:    1,843,211,008 B sent
```

Tip

`psutil` offers one API on Linux, Windows and macOS instead of parsing `/proc` or perfmon output — useful for exporting metrics from inside a service.

CPU Time vs Wall Time

cpu_vs_wall.py

```python
import time
from collections.abc import Callable


def measure(label: str, task: Callable[[], object]) -> None:
    """Compare CPU time with elapsed (wall-clock) time."""
    wall, cpu = time.perf_counter(), time.process_time()
    task()
    wall = time.perf_counter() - wall
    cpu = time.process_time() - cpu
    print(f"{label:<6} wall {wall:.2f} s  cpu {cpu:.2f} s  "
          f"ratio {cpu / wall:.0%}")


measure("crunch", lambda: sum(i * i for i in range(10_000_000)))
measure("sleep", lambda: time.sleep(0.5))
```

SAMPLE OUTPUT

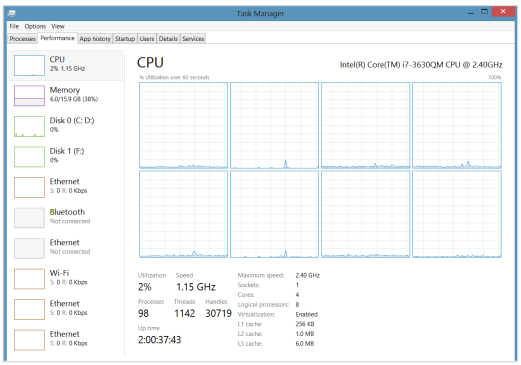
```
crunch wall 0.58 s  cpu 0.58 s  ratio 100%
sleep  wall 0.50 s  cpu 0.00 s  ratio 0%
```

Standard library	Tells you
`time.perf_counter()`	Elapsed wall-clock time
`time.process_time()`	CPU time of this process
`os.process_cpu_count()`	CPUs this process may use (3.13+)
`resource.getrusage()`	CPU time, peak RSS, switches (Unix)
`os.getloadavg()`	Load averages (Unix)

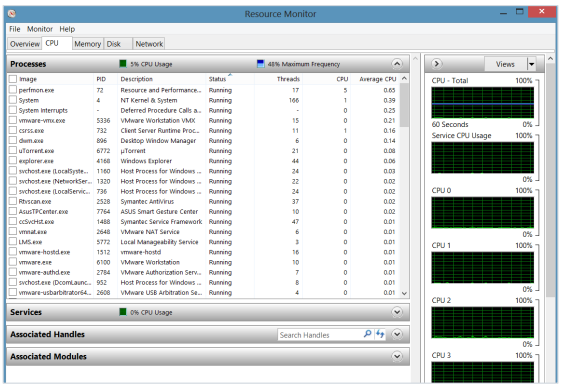
Key idea

CPU ÷ wall ≈ 100%: CPU-bound. ≈ 0%: waiting for I/O, sleep or locks.

Windows: Task Manager and Resource Monitor



Task Manager → Performance: per-CPU graphs, processes, threads, handles, cache sizes

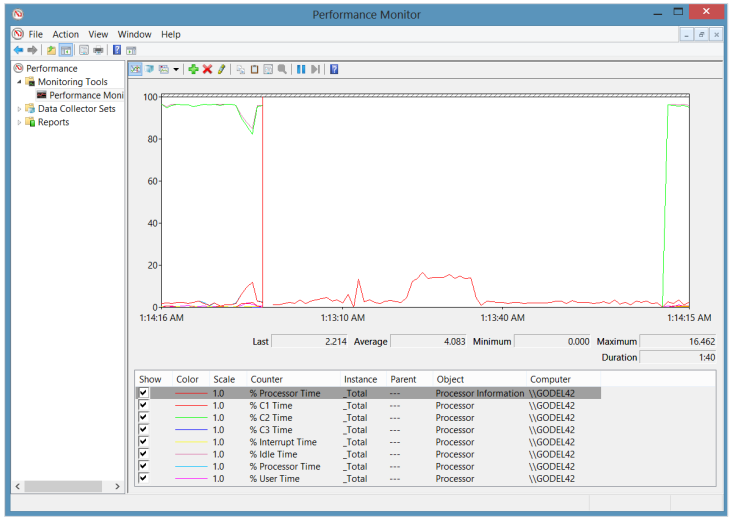


Resource Monitor (`resmon`) → CPU: per-process CPU, threads, current frequency

Tip

Windows 10/11 arrange the panels differently but show the same data. Right-click the CPU graph → *Change graph to* → *Logical processors* for per-core graphs.

Windows: Performance Monitor



- `perfmon`: live graphs of any performance counter
- Counter path: `\Object(Instance)\Counter`, e.g. `\Processor(_Total)\% User Time`
- Data Collector Sets record `.blg` logs for later analysis
- Same counters from the command line: `typeperf`, `logman`, PowerShell `Get-Counter`

Processor Power Management

ACPI state	What it controls	Effect on measurements
P-states (performance)	Frequency and voltage while running	The same code runs faster or slower
C-states (idle)	Sleep depth of idle cores	Waking from deep sleep adds latency
T-states (throttling)	Clock throttling, mainly thermal	Sudden slow-downs when hot

TERMINAL — WINDOWS (C:\>) AND LINUX ($)

```
C:\> powercfg /list
C:\> powercfg /qh scheme_current sub_processor
$ cpupower frequency-info
$ sudo cpupower frequency-set -g performance
```

Tip

Benchmark with a fixed power plan (Windows: High performance; Linux: `performance` governor): frequency scaling can change timings by 2× or more.

typeperf: Counters from the Command Line

COMMAND PROMPT — SAMPLE OUTPUT

```
C:\> typeperf "\Processor(_Total)\% Privileged Time" "\Processor(_Total)\% User Time" -si 1 -sc 3

"(PDH-CSV 4.0)","\\LAB-PC\Processor(_Total)\% Privileged Time","\\LAB-PC\Processor(_Total)\% User Time"
"09/27/2026 10:58:01.214","1.953125","12.500000"
"09/27/2026 10:58:02.229","1.562500","13.281250"
"09/27/2026 10:58:03.241","2.343750","12.890625"

The command completed successfully.
```

COMMAND PROMPT — COUNTERS FROM A FILE, OUTPUT TO CSV

```
C:\> type cpu-util.txt
\Processor(_Total)\% Privileged Time
\Processor(_Total)\% User Time
C:\> typeperf -cf cpu-util.txt -si 5 -sc 720 -o cpu.csv
C:\> typeperf -q Processor
```

logman and relog: Recording Counters

COMMAND PROMPT — SAMPLE SESSION

```
C:\> logman create counter py-cpu -f bincirc -max 250 -si 00:00:05 ^
More? -c "\Processor(_Total)\% Processor Time" -o C:\perflogs\py-cpu
The command completed successfully.
C:\> logman start py-cpu
The command completed successfully.
     ... run the Python workload ...
C:\> logman stop py-cpu
The command completed successfully.
C:\> relog C:\perflogs\py-cpu.blg -f csv -o cpu.csv
```

- `logman` creates, starts and stops Performance Monitor data collectors from the command line
- `bincirc`: binary circular log, at most 250 MB — safe to leave running
- `relog` converts `.blg` logs to CSV (or filters and resamples them)

QUIZ

Quick Check: Windows Counters

On Windows, which counter shows the share of CPU time spent in kernel mode?

A `\Processor(_Total)\% User Time`

B `\Processor(_Total)\% Privileged Time`

C `\Processor(_Total)\% Idle Time`

D `\Memory\Pages/sec`

ANSWER

Quick Check: Windows Counters

On Windows, which counter shows the share of CPU time spent in kernel mode?

A `\Processor(_Total)\% User Time`

B `\Processor(_Total)\% Privileged Time`

C `\Processor(_Total)\% Idle Time`

D `\Memory\Pages/sec`

Why

Windows calls kernel mode "privileged" mode: `% Privileged Time` corresponds to sy in `vmstat`, `% User Time` to us.

05

Measuring Python Code

Benchmark to know how fast; profile to know where the time goes

- Timing code with `time.perf_counter` and `timeit`
- Function-level profiling with `cProfile`, `pstats` and SnakeViz
- Line and memory profilers: `line_profiler`, `tracemalloc`, memray
- Sampling profilers, `pytest-benchmark` and a profiling workflow

Make It Work, Make It Right, Make It Fast



Make it work
correct results, passing tests

Make it right
clean design, readable code

Make it fast
measure, then fix the real hot spots

> ★ **Rule of thumb**
> “Premature optimization is the root of all evil.” — Donald Knuth, 1974

- Finding the slow part is the single most important step
- Optimise only against a **measurable goal**: the NFRs of Section 2
- Most run time is spent in a small fraction of the code: the **hot spots**

Benchmarking vs Profiling

	Benchmarking	Profiling
Question	*How long* does it take?	*Where* do time and memory go?
Result	one number: time, throughput, memory	a breakdown per function or per line
Scope	a statement, a function, a whole program	everything the program executes
Tools	`perf_counter`, `timeit`, `pytest-benchmark`	`cProfile`, `line_profiler`, `tracemalloc`, `py-spy`
Use it to	compare versions, catch regressions	find the hot spot worth optimising

> ★ **Key idea**
> Benchmark to know **whether** you improved; profile to know **what** to improve.

Timing Code with time.perf_counter

timing.py

```python
import time


def build_squares(n: int) -> list[int]:
    return [i * i for i in range(n)]


wall_start = time.perf_counter()
cpu_start = time.process_time()
squares = build_squares(1_000_000)
time.sleep(0.2)               # simulated I/O wait
wall = time.perf_counter() - wall_start
cpu = time.process_time() - cpu_start
print(f"wall: {wall * 1000:.1f} ms, cpu: {cpu * 1000:.1f} ms")
```

SAMPLE OUTPUT — TIMINGS VARY BY MACHINE

```
wall: 247.7 ms, cpu: 46.9 ms
```

Clock	Measures
`perf_counter()`	elapsed (wall) time; monotonic, highest resolution: use for durations
`process_time()`	CPU time of this process; excludes sleeping and waiting
`time()`	calendar time; can jump when the clock is adjusted

Tip

Wall time − CPU time ≈ time spent waiting (I/O, locks, other processes).

timeit on the Command Line

SAMPLE OUTPUT — TIMINGS VARY BY MACHINE

```
$ python -m timeit "sum(range(1000))"
50000 loops, best of 5: 6.93 usec per loop
$ python -m timeit -s "data = list(range(1000))" "sum(data)"
100000 loops, best of 5: 3.33 usec per loop
$ python -m timeit -s "data = list(range(1000))" "total = 0" "for x in data: total += x"
20000 loops, best of 5: 17 usec per loop
```

Option	Meaning
`-s SETUP`	run once before timing; not included in the result
`-n N`	loops per repeat (default: calibrated until one repeat takes ≥ 0.2 s)
`-r R`	number of repeats (default 5); the best one is reported
`-u UNIT`	output unit: `nsec`, `usec`, `msec` or `sec`
`-v`	verbose: also print the raw time of every repeat

timeit in Code: repeat and min

bench_sum.py

```python
import timeit


def loop_sum(values: list[int]) -> int:
    total = 0
    for value in values:
        total += value
    return total


data = list(range(10_000))
for label, func in [("for loop", loop_sum), ("sum()", sum)]:
    times = timeit.repeat(lambda: func(data),
                          number=200, repeat=5)
    best = min(times) / 200
    print(f"{label:<9}{best * 1e6:7.1f} µs per call")
```

SAMPLE OUTPUT — TIMINGS VARY BY MACHINE

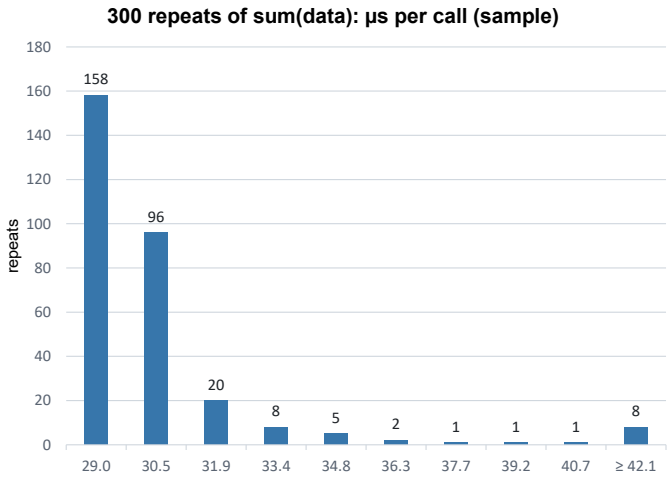
```
for loop   151.5 µs per call
sum()       30.1 µs per call
```

- `timeit(stmt, number=n)`: total seconds for `n` runs
- `repeat(stmt, number=n, repeat=r)`: a list of `r` totals
- `stmt`: a callable or a string (+ `setup=`, `globals=`)
- Divide by `number`, then take the **minimum**

Tip

Time the function you care about, with realistic input sizes.

Why Report the Minimum of Repeats?



- Noise only ever **adds** time: other processes, interrupts, CPU frequency changes
- The fastest run is the one least disturbed
- Mean and maximum describe the machine, not the code

Key idea

For end-to-end latency report percentiles (p50, p95, p99), as in Section 1. For micro-benchmarks report the minimum.

QUIZ

Quick Check: Reporting timeit Results

`timeit.repeat(stmt, number=1000, repeat=5)` returned `[0.82, 0.79, 0.95, 0.80, 1.40]` (seconds). What is the best estimate of the cost of one execution of `stmt`?

- **A** 0.79 ms: the minimum divided by `number`
- **B** 0.95 ms: the mean divided by `number`
- **C** 1.40 ms: the worst case divided by `number`
- **D** 0.79 s: the minimum repeat

ANSWER

Quick Check: Reporting timeit Results

`timeit.repeat(stmt, number=1000, repeat=5)` returned `[0.82, 0.79, 0.95, 0.80, 1.40]` (seconds). What is the best estimate of the cost of one execution of `stmt`?

- **A** 0.79 ms: the minimum divided by `number`
- **B** 0.95 ms: the mean divided by `number`
- **C** 1.40 ms: the worst case divided by `number`
- **D** 0.79 s: the minimum repeat

Why

Each value is the total for 1,000 executions, so divide by `number`. The minimum is the least disturbed repeat; the 1.40 s outlier inflates the mean and says nothing about the statement itself.

cProfile: The Built-in Deterministic Profiler

Deterministic
- Hooks **every** call and return
- Exact call counts
- Time per function

Built in
- Standard library: nothing to install
- `python -m cProfile app.py`
- or `cProfile.Profile` in code

Function level
- Includes C functions such as `str.strip`
- Cannot see single lines
- Costs a little time per call

Module	Written in	Overhead	Use it
`cProfile`	C	low for most programs	default choice
`profile`	pure Python	very high (up to ~80× measured)	only to extend the profiler

The Example Program: words.py

words.py

```python
def normalize(word: str) -> str:
    return word.strip().lower()


def unique_words(words: list[str]) -> list[str]:
    seen: list[str] = []
    for word in words:
        word = normalize(word)
        if word not in seen:
            seen.append(word)
    return seen


def main() -> None:
    words = [f" Word{i % 5000} " for i in range(20_000)]
    print(len(unique_words(words)), "unique words")


if __name__ == "__main__":
    main()
```

OUTPUT

```
5000 unique words
```

- Normalises 20,000 words and keeps the first occurrence of each
- Runs in about 0.37 s (sample) — slow for so little data
- Two candidates: `normalize` is called 20,000 times; `unique_words` once

Guess first

Before profiling, write down your guess. Where does the time go?

cProfile from the Command Line

SAMPLE OUTPUT — TIMINGS VARY BY MACHINE (FIRST ROWS)

```
$ python -m cProfile -s tottime words.py
5000 unique words
         65015 function calls in 0.377 seconds

   Ordered by: internal time

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.364    0.364    0.374    0.374 words.py:5(unique_words)
    20000    0.005    0.000    0.009    0.000 words.py:1(normalize)
    20000    0.002    0.000    0.002    0.000 {method 'strip' of 'str' objects}
        1    0.002    0.002    0.376    0.376 words.py:14(main)
    20000    0.002    0.000    0.002    0.000 {method 'lower' of 'str' objects}
     5000    0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
```

Tip

`-s tottime|cumulative|calls` sorts the table; `-o words.prof` saves binary results for `pstats` or SnakeViz; `-m package.module` profiles a module.

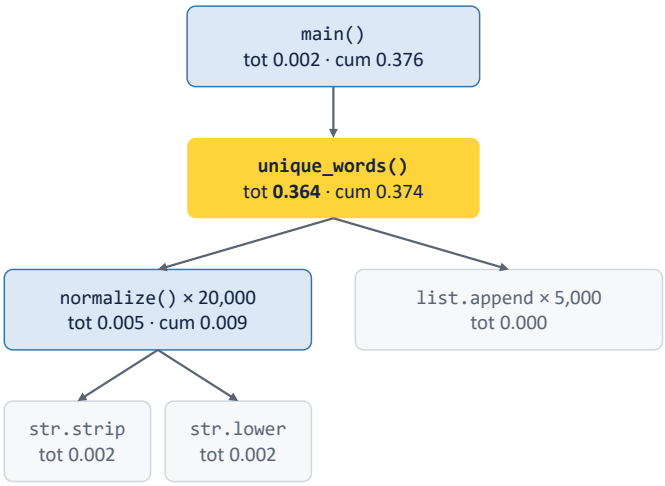
Reading a pstats Table

Column	Meaning
`ncalls`	number of calls; `3/1` = 3 calls in total, 1 primitive (not recursive)
`tottime`	time spent **in the function itself**, excluding the functions it calls
`percall`	`tottime / ncalls`
`cumtime`	cumulative time **including** all sub-calls (the whole subtree)
`percall`	`cumtime /` primitive calls
`filename:lineno(function)`	where the function is defined; `{...}` marks a built-in C function

Key idea

High `tottime` → the function's own code is slow (a hot spot). High `cumtime` but low `tottime` → the time is spent in what it calls.

tottime vs cumtime in the Call Tree



cumtime = tottime + cumtime of all callees

`unique_words`: 0.364 + 0.009 + 0.000 ≈ 0.374

- Its 0.364 s of own time is not spent in any call
- It is spent in `word not in seen`: an operator, not a function call
- cProfile cannot say which line → `line_profiler`

Profiling a Region with cProfile.Profile

profile_words.py

```python
import cProfile
import pstats

from words import unique_words

words = [f" Word{i % 5000} " for i in range(20_000)]

with cProfile.Profile() as profiler:
    unique_words(words)

stats = pstats.Stats(profiler).strip_dirs()
stats.sort_stats("cumulative").print_stats(3)
stats.dump_stats("words.prof")
```

- `with cProfile.Profile() as profiler:` profiles only the block
- `pstats.Stats` sorts, filters and prints
- `dump_stats` saves the data for later analysis
- One-liner: `cProfile.run("main()", sort="tottime")`
- Manual: `profiler.enable()` … `profiler.disable()`

5 · MEASURING PYTHON CODE

Exploring Results with pstats

explore.py

```python
import pstats

stats = pstats.Stats("words.prof").strip_dirs()
stats.sort_stats("tottime").print_stats(10)   # top 10 rows
stats.print_stats("words")                    # regex filter
stats.print_callers("normalize")              # who calls it?
stats.print_callees("unique_words")           # what does it call?
```

SAMPLE OUTPUT — TIMINGS VARY BY MACHINE (PRINT_CALLERS AND PRINT_CALLEES)

```
Function              was called by...
                          ncalls  tottime  cumtime
words.py:1(normalize)  <-   20000    0.005    0.010  words.py:5(unique_words)

Function                 called...
                             ncalls  tottime  cumtime
words.py:5(unique_words)  ->   20000    0.005    0.010  words.py:1(normalize)
                                5000    0.001    0.001  {method 'append' of 'list' objects}
```

Tip

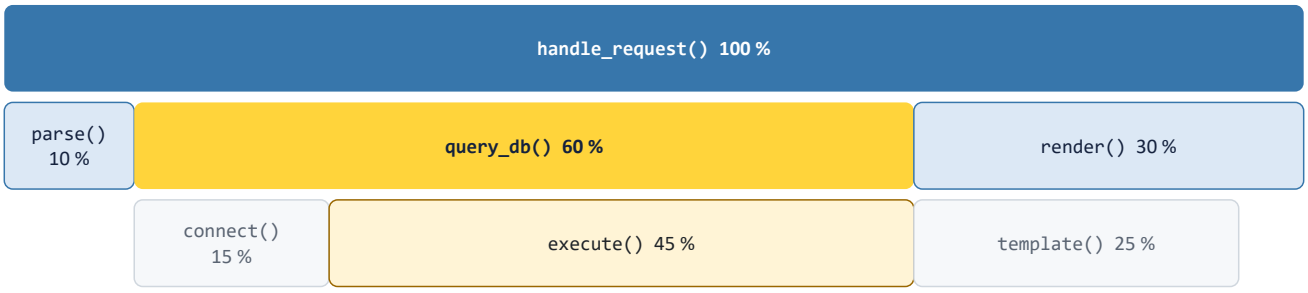
Sort keys: "tottime" finds hot spots, "cumulative" expensive call paths, "calls" functions called surprisingly often.

5 · MEASURING PYTHON CODE

Visualising Profiles with SnakeViz

TERMINAL

```
$ pip install snakeviz
$ python -m cProfile -o app.prof app.py
$ snakeviz app.prof          # opens an interactive chart in the browser
```



Icicle chart (illustrative): width = cumulative time, depth = call stack. Wide boxes without children are hot spots.

QUIZ

Quick Check: Read the Profile

A billing script takes 1.8 s under the profiler. Which **single** change gives the biggest speed-up?

```
 ncalls  tottime  percall  cumtime  percall filename:lineno(function)
  20000    1.143    0.000    1.709    0.000 billing.py:4(load_tax_table)
4000000    0.474    0.000    0.474    0.000 {method 'split' of 'str' objects}
  20000    0.092    0.000    0.092    0.000 {method 'splitlines' of 'str' objects}
      1    0.058    0.058    1.810    1.810 billing.py:21(main)
  20000    0.012    0.000    1.738    0.000 billing.py:12(price_with_tax)
```

A. Speed up `main`: it has the largest `cumtime`

B. Load the tax table once instead of once per order

C. Replace `str.split`: it is called 4,000,000 times

D. Speed up `price_with_tax`: its `cumtime` is 1.738 s

ANSWER

Quick Check: Read the Profile

A billing script takes 1.8 s under the profiler. Which **single** change gives the biggest speed-up?

```
 ncalls  tottime  percall  cumtime  percall filename:lineno(function)
  20000    1.143    0.000    1.709    0.000 billing.py:4(load_tax_table)
4000000    0.474    0.000    0.474    0.000 {method 'split' of 'str' objects}
  20000    0.092    0.000    0.092    0.000 {method 'splitlines' of 'str' objects}
      1    0.058    0.058    1.810    1.810 billing.py:21(main)
  20000    0.012    0.000    1.738    0.000 billing.py:12(price_with_tax)
```

A. Speed up `main`: it has the largest `cumtime`

B. Load the tax table once instead of once per order

C. Replace `str.split`: it is called 4,000,000 times

D. Speed up `price_with_tax`: its `cumtime` is 1.738 s

★ **Why**

`load_tax_table` has the highest own time and, with its callees `split` and `splitlines`, accounts for 1.71 of 1.81 s. It runs 20,000 times but returns the same table every time: load it once (or cache it).

A Reusable Profiling Decorator

profiling.py

```python
import cProfile
import functools
import pstats


def profiled(func):
    """Print a cProfile report after every call of func."""
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        with cProfile.Profile() as profiler:
            result = func(*args, **kwargs)
        stats = pstats.Stats(profiler).strip_dirs()
        stats.sort_stats("tottime").print_stats(5)
        return result
    return wrapper
```

- The decorator pattern from Module 2
- Profiles every call of the decorated function
- Prints the top five rows by own time

Watch out

Do not call it `profile`: that name shadows the standard-library module and clashes with line_profiler's `@profile`.

Profiling Bubble Sort with the Decorator

sorting.py

```python
import random

from profiling import profiled


@profiled
def bubble_sort(items: list[int]) -> None:
    n = len(items)
    for i in range(n):
        for j in range(n - i - 1):
            if items[j] > items[j + 1]:
                items[j], items[j + 1] = items[j + 1], items[j]


data = [random.randint(0, 1000) for _ in range(2_000)]
bubble_sort(data)
```

SAMPLE (SHORTENED)

```
4 function calls in 0.121 seconds

ncalls tottime cumtime function
     1   0.121   0.121 bubble_sort
     1   0.000   0.000 {method 'disable'}
     1   0.000   0.000 __exit__
     1   0.000   0.000 {built-in len}
```

Key idea

All 0.121 s are own time of one call: cProfile shows **which function**, not **which line**.

line_profiler: Time per Line

TERMINAL

```
$ pip install line_profiler
$ kernprof -l -v words.py        # after adding @profile to unique_words
```

SAMPLE OUTPUT — TIMINGS VARY BY MACHINE (TIMES IN MICROSECONDS)

```
Total time: 0.400567 s
File: words.py
Function: unique_words at line 5

Line #      Hits         Time  Per Hit   % Time  Line Contents
==============================================================
     5                                           @profile
     6                                           def unique_words(words):
     7         1          1.1      1.1      0.0      seen = []
     8     20001       2915.0      0.1      0.7      for word in words:
     9     20000       7480.3      0.4      1.9          word = normalize(word)
    10     20000     389120.4     19.5     97.1          if word not in seen:
    11      5000       1050.2      0.2      0.3              seen.append(word)
    12         1          0.4      0.4      0.0      return seen
```


Measuring Memory with tracemalloc

peak_memory.py

```python
import tracemalloc
from collections.abc import Callable


def peak_kib(func: Callable[[], object]) -> float:
    """Run func; return its peak traced memory in KiB."""
    tracemalloc.start()
    func()
    peak = tracemalloc.get_traced_memory()[1]
    tracemalloc.stop()
    return peak / 1024


n = 1_000_000
eager = peak_kib(lambda: sum([x * x for x in range(n)]))
lazy = peak_kib(lambda: sum(x * x for x in range(n)))
print(f"list comprehension: {eager:10,.1f} KiB")
print(f"generator:          {lazy:10,.1f} KiB")
```

SAMPLE OUTPUT — TIMINGS VARY BY MACHINE

```
list comprehension:   39,499.7 KiB
generator:                0.5 KiB
```

- Standard library, no installation
- `start()` / `stop()` tracing
- `get_traced_memory()` → `(current, peak)` bytes
- `take_snapshot()` → allocations per line

Note

Counts memory allocated by Python code, not the whole process size (RSS) that `top` or Task Manager show (Section 4).

tracemalloc Snapshots: Where Memory Goes

users.py

```python
import tracemalloc

tracemalloc.start()
names = [f"user{i:06}" for i in range(100_000)]
scores = {name: len(name) * 1.5 for name in names}
snapshot = tracemalloc.take_snapshot()
tracemalloc.stop()

for stat in snapshot.statistics("lineno")[:2]:
    line = stat.traceback[0].lineno
    print(f"line {line}: {stat.size / 1024:,.0f} KiB"
          f" in {stat.count:,} blocks")
```

SAMPLE OUTPUT — TIMINGS VARY BY MACHINE

```
line 5: 6,099 KiB in 99,997 blocks
line 4: 5,763 KiB in 100,001 blocks
```

finding a leak

```python
before = tracemalloc.take_snapshot()
run_suspicious_code()
after = tracemalloc.take_snapshot()
top = after.compare_to(before, "lineno")
for diff in top[:3]:
    print(diff)       # size and count +/-
```

Tip

Memory that keeps growing between two snapshots of a long-running program is a leak candidate.

Line-by-Line Memory with memory_profiler

TERMINAL

```
$ pip install memory_profiler
$ python -m memory_profiler tables.py      # functions marked @profile
```

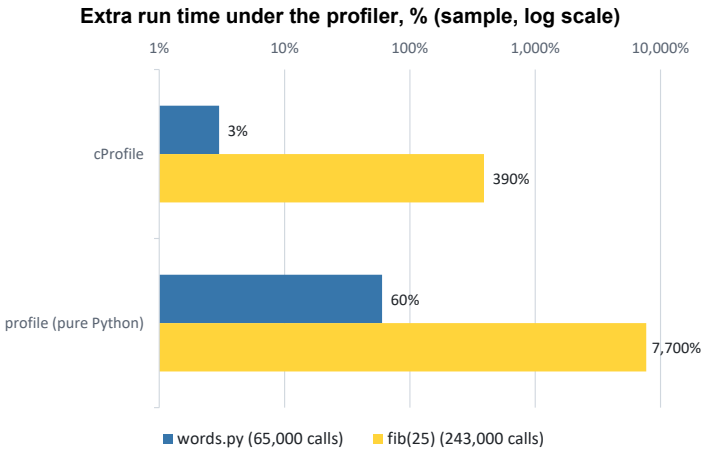
SAMPLE OUTPUT — TIMINGS VARY BY MACHINE

```
Line #    Mem usage    Increment  Occurrences   Line Contents
=============================================================
     4     21.3 MiB     21.3 MiB           1   @profile
     5                                         def build_tables() -> int:
     6     29.0 MiB      7.6 MiB           1       zeros = [0] * 1_000_000
     7     60.4 MiB     31.4 MiB           1       floats = list(map(float, range(1_000_000)))
     8     60.4 MiB      0.0 MiB           1       total = sum(floats)
     9     29.9 MiB    -30.5 MiB           1       del floats
```

Status

memory_profiler is no longer maintained (last release 2022). For new work use **memray** (Linux/macOS, native allocations, flame graphs) or **Scalene** (CPU and memory per line).

Profiler Overhead: The Observer Effect



- Deterministic profilers pay a cost **per call**
- Call-heavy code is distorted most
- Tiny functions look more expensive than they are

Key idea

Use the profiler to find **where**; use benchmarks without the profiler to measure **how much** you gained.

Sampling Profilers: py-spy

	Deterministic (cProfile)	Sampling (py-spy, Scalene)
How	records every call and return	reads the call stack ~100 times per second
Overhead	per call: up to several ×	small and constant
Counts	exact call counts	statistical: no call counts
Setup	run the program under it	attach to a **running** process

TERMINAL

```
$ pip install py-spy
$ py-spy record -o profile.svg -- python words.py   # flame graph
$ py-spy top --pid 12345                           # live, like top
$ py-spy dump --pid 12345                          # stack of every thread
```

Tip

Safe for production: the target process is neither modified nor restarted.

Benchmarks as Tests with pytest-benchmark

test_words.py

```python
from words import unique_words, unique_words_set

WORDS = [f" Word{i % 5000} " for i in range(20_000)]


def test_unique_list(benchmark):
    result = benchmark(unique_words, WORDS)
    assert len(result) == 5000

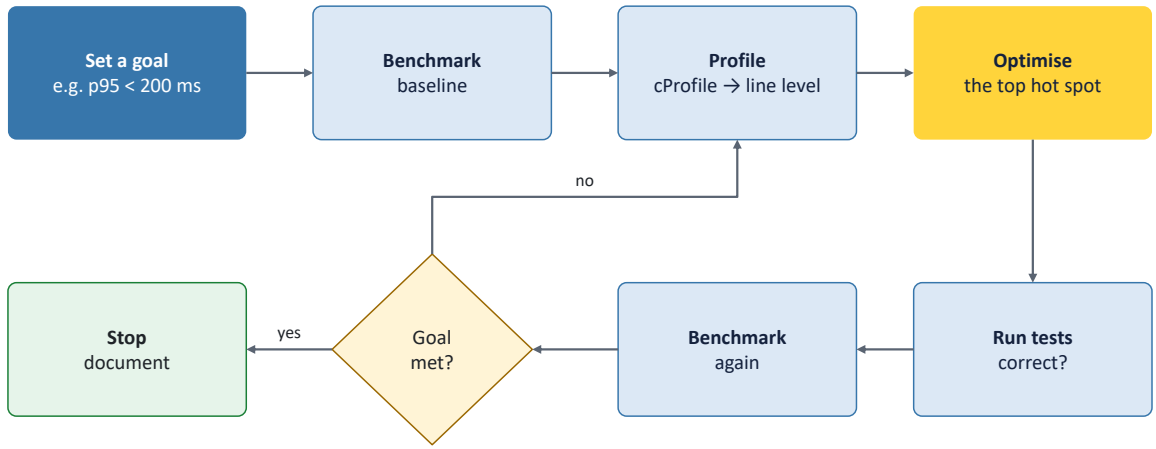

def test_unique_set(benchmark):
    result = benchmark(unique_words_set, WORDS)
    assert len(result) == 5000
```

- `benchmark` is a pytest **fixture** (Module 3)
- Calibrates rounds and iterations automatically
- Still an ordinary test: the `assert` checks correctness
- `--benchmark-autosave`, `--benchmark-compare` catch regressions

SAMPLE OUTPUT — TIMINGS VARY BY MACHINE

```
$ pytest test_words.py --benchmark-columns=min,mean,rounds
Name (time in ms)          Min                 Mean            Rounds
test_unique_set         2.9021 (1.0)          3.0958 (1.0)          317
test_unique_list      354.5169 (122.16)     365.2081 (117.97)         5
```


A Profiling Workflow



Tip

One change per iteration, so you know which change helped.

5 · MEASURING PYTHON CODE

Profiling Best Practices

Realistic conditions
- Production-like data sizes
- Same Python version and build

Profile the hot path
- Critical sections first
- Representative requests, not idle time

Compare fairly
- Same machine, repeated runs
- One change at a time

Automate
- `pytest-benchmark` in CI
- Fail on regressions

Monitor production
- Metrics and tracing (Sections 2 and 4)
- Prometheus, OpenTelemetry

Stay readable
- Keep clear code where speed is irrelevant
- Comment each optimisation

DISCUSS

Which Tool Would You Use?

#	Situation
1	Is `" ".join(parts)` faster than `+=` for 100 short strings?
2	The nightly report takes 40 min; you have no idea why
3	One 200-line function is the hot spot
4	A long-running service grows by 50 MB per hour
5	The production API is slow right now; you cannot restart it
6	Make sure nobody makes `parse()` slower again

ANSWER

Which Tool Would You Use?

#	Tool	Why
1	`timeit`	micro-benchmark of two statements
2	`cProfile` (+ SnakeViz)	function-level overview of the whole run
3	`line_profiler`	time per line inside one function
4	`tracemalloc` snapshots (or memray)	compare snapshots over time: leak
5	`py-spy` (`top`, `dump`, `record`)	attaches to a running process
6	`pytest-benchmark`	benchmark as a test, compared in CI

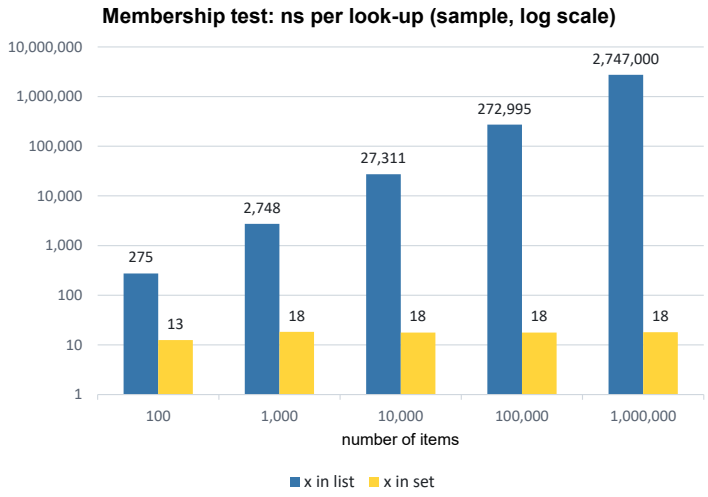
06

Pure-Python Optimizations

Better algorithms first, then cheaper operations

- Complexity of `list`, `deque`, `set` and `dict`; searching with `bisect`
- Built-ins, comprehensions, `join` and local names
- Exceptions, `lru_cache`, `__slots__` and avoiding repeated work
- Imports, logging and resource handling

Algorithms Beat Micro-Optimisations



- A list compares item by item: **O(n)**
- A set hashes the value: **O(1)** on average
- At one million items the set is ~150,000× faster

Key idea

A better algorithm or data structure changes the **growth rate**; micro-optimisations only change the constant factor.

Time Complexity of Built-in Collections

Operation	list	deque	set	dict
add at the end: `append`, `add`, `d[k] = v`	O(1)*	O(1)	O(1)†	O(1)†
add or remove at the front	O(n)	O(1)	—	—
remove from the end: `pop()`	O(1)	O(1)	—	—
access by position: `c[i]`	O(1)	O(n)‡	—	—
membership: `x in c`	O(n)	O(n)	O(1)†	O(1)† keys
delete a given value or key	O(n)	O(n)	O(1)†	O(1)†
sort	O(n log n)	—	—	—

** amortised · † average case (worst case O(n)) · ‡ O(1) at both ends. Source: Python wiki, TimeComplexity.*

Choosing the Right Container

`set` / `frozenset`
- Membership tests, removing duplicates
- `if user in banned:`

`dict`
- Look up values by key
- Index a list once, query it many times

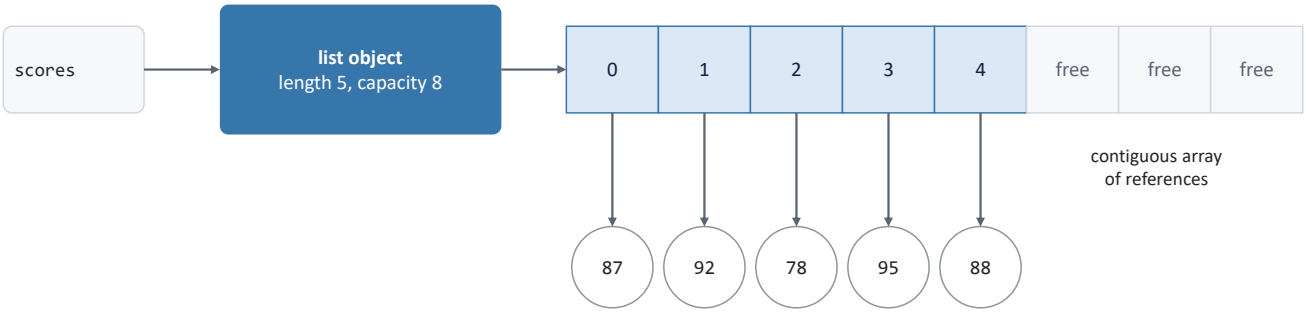
`collections.deque`
- FIFO queues, sliding windows
- append / `popleft` in O(1)

`Counter`, `defaultdict`
- Counting and grouping
- `counts[item] += 1` without key checks

`heapq`
- Top-k and priority queues
- `heapq.nlargest(10, scores)`

`bisect`
- Search and insert in sorted lists
- O(log n) look-ups

Lists Are Dynamic Arrays



SAMPLE OUTPUT — TIMINGS VARY BY MACHINE

```
$ python -m timeit -s "items = [float(i) for i in range(10)]" "items[5]"
20000000 loops, best of 5: 10.2 nsec per loop
$ python -m timeit -s "items = [float(i) for i in range(10_000_000)]" "items[5_000_000]"
20000000 loops, best of 5: 13.4 nsec per loop
```

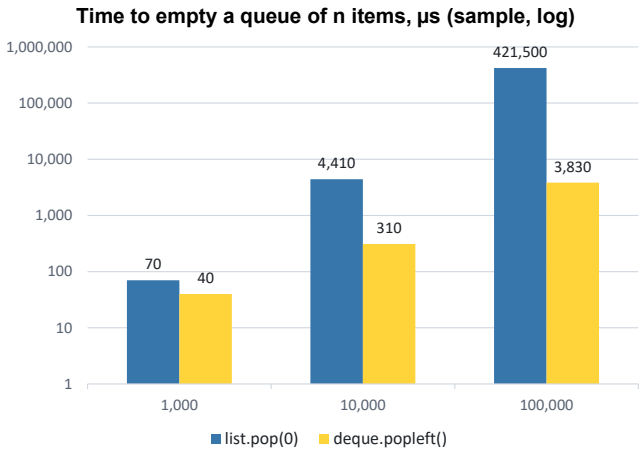
deque: O(1) at Both Ends

queues.py

```python
from collections import deque


def drain_list(n: int) -> None:
    queue = list(range(n))
    while queue:
        queue.pop(0)      # shifts all others


def drain_deque(n: int) -> None:
    queue = deque(range(n))
    while queue:
        queue.popleft()   # O(1)
```



Linear Search: O(n)

search.py

```python
def linear_search(needle: int, items: list[int]) -> int:
    """Return the index of needle in items, or -1."""
    for index, item in enumerate(items):
        if item == needle:
            return index
    return -1


print(linear_search(42, [7, 3, 42, 9, 15]))
print(linear_search(5, [7, 3, 42, 9, 15]))
```

OUTPUT

```
2
-1
```

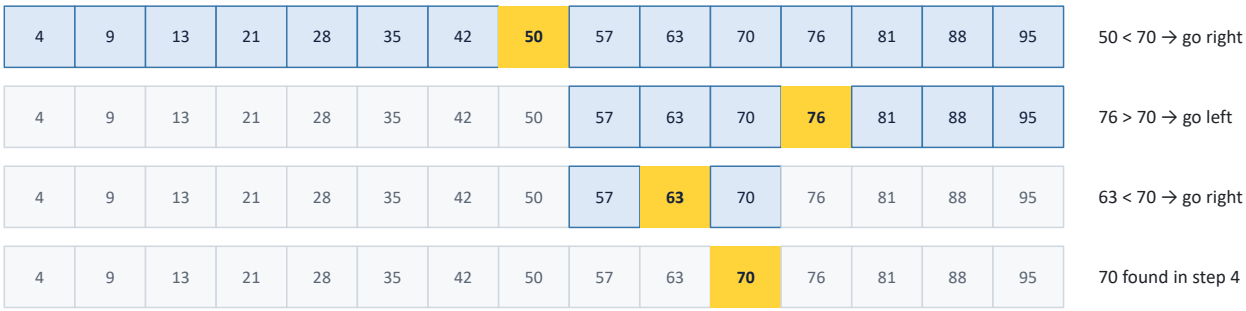


3 comparisons to find 42

- Works on unsorted data
- Worst case n comparisons, n/2 on average
- `x in items` and `items.index(x)` do the same scan in C

6 · PURE-PYTHON OPTIMIZATIONS

Binary Search: Halve the Range Each Step



Key idea

Requires **sorted** data. Each comparison halves the range: 1,000,000 items need at most 20 comparisons ($\log_2 n$), a linear search up to 1,000,000.

FIND THE BUG

Find the Bug in This Binary Search

from the original slides

```python
def binary_search(needle, haystack):
    imin, imax = 0, len(haystack)
    while True:
        if imin > imax:
            return -1
        midpoint = (imin + imax) // 2
        if haystack[midpoint] > needle:
            imax = midpoint
        elif haystack[midpoint] < needle:
            imin = midpoint + 1
        else:
            return midpoint
```

It finds every value that is present. What happens with these two calls, and why?

`binary_search(2, [1, 3])` and `binary_search(5, [1, 3])`

ANSWER

Find the Bug in This Binary Search

- `binary_search(2, [1, 3])` **never returns**: `imin == imax == 1`, line 8 keeps setting `imax = 1`
- `binary_search(5, [1, 3])` raises `IndexError`: `midpoint == len(haystack)`
- Cause: `imax` is exclusive, so the loop must stop when `imin == imax`

```
def binary_search(needle: int, haystack: list[int]) -> int:
    low, high = 0, len(haystack)        # search in [low, high)
    while low < high:
        mid = (low + high) // 2
        if haystack[mid] < needle:
            low = mid + 1
        elif haystack[mid] > needle:
            high = mid
        else:
            return mid
    return -1


print([binary_search(n, [1, 3, 5, 7]) for n in (0, 1, 2, 5, 7, 8)])
```

OUTPUT

```
[-1, 0, -1, 2, 3, -1]
```


6 · PURE-PYTHON OPTIMIZATIONS

Binary Search with the bisect Module

bisect_demo.py

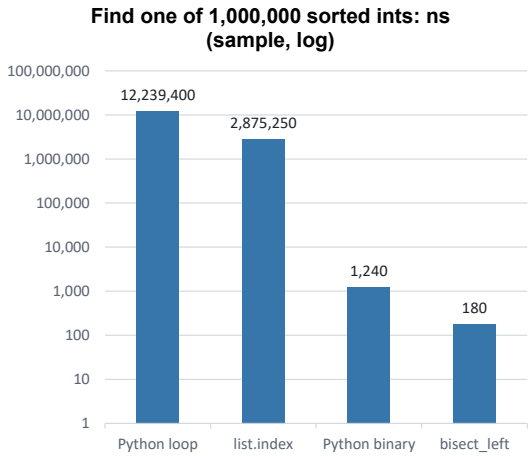
```
from bisect import bisect_left, insort


def contains(items: list[int], value: int) -> bool:
    """Binary search in a sorted list: O(log n)."""
    index = bisect_left(items, value)
    return index < len(items) and items[index] == value


scores = [52, 61, 67, 74, 78, 85, 91]   # sorted
print(contains(scores, 74), contains(scores, 75))
insort(scores, 70)                      # stays sorted
print(scores)
```

OUTPUT

```
True False
[52, 61, 67, 70, 74, 78, 85, 91]
```



QUIZ

Quick Check: List Complexity

Which operation on a Python `list` of n items takes **O(n)** time?

A `items.append(x)`

B `items[-1]`

C `items.pop()`

D `items.insert(0, x)`

ANSWER

Quick Check: List Complexity

Which operation on a Python `list` of n items takes **O(n)** time?

A `items.append(x)`

B `items[-1]`

C `items.pop()`

D `items.insert(0, x)`

★ **Why**

Inserting at the front shifts every existing reference one slot to the right. Appending, indexing and popping from the end touch only one slot: O(1) (append is amortised O(1)).

Lists vs Tuples

	`list`	`tuple`
Mutability	mutable, can grow	immutable
Size of 5 ints	104 bytes, spare slots once grown	88 bytes, exact
Literal creation	~31 ns: built at run time	~5 ns: one constant
Hashable	no	yes, if its items are
Typical use	a growing collection of similar items	a fixed record: `(x, y)`, dict keys

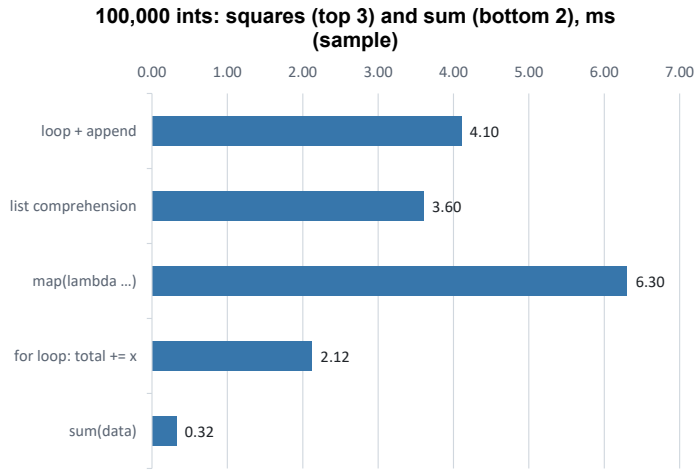
SAMPLE OUTPUT — TIMINGS VARY BY MACHINE

```
$ python -m timeit "[1, 2, 3, 4, 5]"
10000000 loops, best of 5: 31.5 nsec per loop
$ python -m timeit "(1, 2, 3, 4, 5)"
50000000 loops, best of 5: 4.57 nsec per loop
```

Common misconception

A tuple **literal** is a constant created once by the compiler. `tuple(range(5))` costs about as much as `list(range(5))`.

Built-in Functions Run in C



- `sum`, `min`, `max`, `sorted`, `any`, `all` loop in C
- One call replaces thousands of bytecode steps
- `sum(data)` is ~6× faster than the loop

Common misconception

`map(lambda x: x**2, data)` is no optimisation: it calls a Python function per item (slower than a comprehension) and returns a lazy iterator, not a list.

Comprehensions, Generators and Laziness

lazy.py

```python
from itertools import count, islice

squares = [x * x for x in range(10)]      # all now
squares_gen = (x * x for x in range(10))  # on demand
first_ten = list(islice(count(), 10))     # lazy, infinite

print(squares[-1], sum(squares_gen), first_ten[-1])
```

OUTPUT

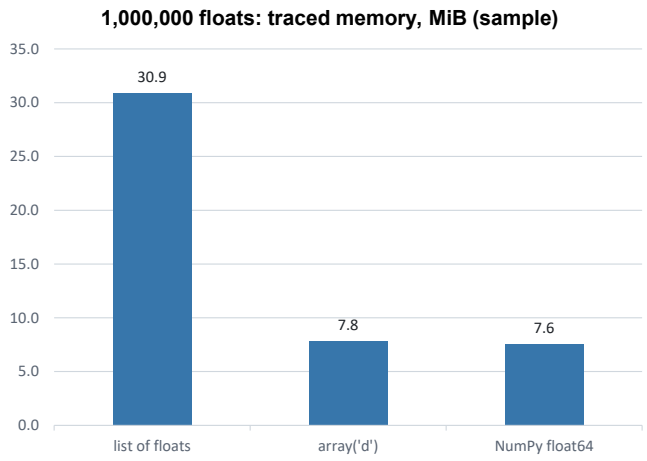
```
81 285 9
```

> **Note**
> Syntax and semantics: Module 2. Here we only measure.

`sum` of 1,000,000 squares	Time	Peak memory
`sum([x * x for x in data])`	69 ms	39 MB
`sum(x * x for x in data)`	65 ms	0.5 KB

- Same speed, very different memory
- List: several passes, indexing, `len()`
- Generator: one pass over large or endless data

Memory-Efficient Data: array and NumPy



- A list holds **references** to 24-byte float objects
- `array` and NumPy store raw 8-byte values side by side
- Avoid copies: list slices copy, NumPy slices are views
- Create fewer temporary objects in hot loops
- Caches that must not keep objects alive: `weakref`

> **Note**
> Garbage-collector tuning with the `gc` module: Section 1.

Building Strings with join

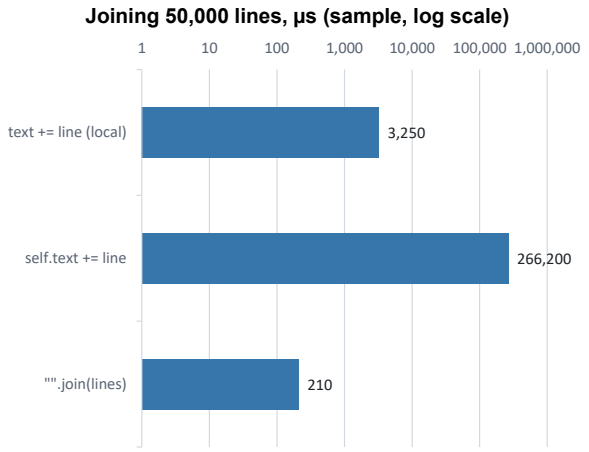
strings.py

```python
def concat(lines: list[str]) -> str:
    text = ""
    for line in lines:
        text += line      # may copy text each time
    return text


def join(lines: list[str]) -> str:
    return "".join(lines)   # one pass, one allocation
```

Watch out

CPython can sometimes extend a string in place, but only while nothing else refers to it. `join` is guaranteed to be linear.



Local Names Are Faster Than Globals

scope_speed.py

```python
import time

start = time.perf_counter()
total = 0
for i in range(5_000_000):      # globals
    total += i % 7
print(f"module level:  {time.perf_counter() - start:.3f} s")


def main() -> None:
    start = time.perf_counter()
    total = 0
    for i in range(5_000_000):  # locals
        total += i % 7
    print(f"in a function: {time.perf_counter() - start:.3f} s")


main()
```

SAMPLE OUTPUT — TIMINGS VARY BY MACHINE

```
module level:  0.388 s
in a function: 0.165 s
```

- Locals live in a fixed array: `LOAD_FAST`
- Globals and built-ins: dictionary look-ups
- Put script code in a `main()` function
- Pass values as arguments instead of reading globals

Python version note

Since 3.11 the interpreter caches global look-ups; aliasing `sqrt = math.sqrt` before a loop now gains only a few percent.

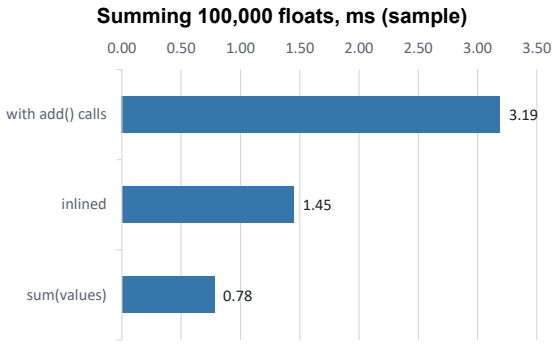
Function Calls Have a Cost

calls.py

```python
def add(a: float, b: float) -> float:
    return a + b


def total_with_calls(values: list[float]) -> float:
    total = 0.0
    for value in values:
        total = add(total, value)   # a call per item
    return total


def total_inlined(values: list[float]) -> float:
    total = 0.0
    for value in values:
        total = total + value
    return total
```



Tip

Inline only in a proven hot loop. Better: let a built-in remove the loop.

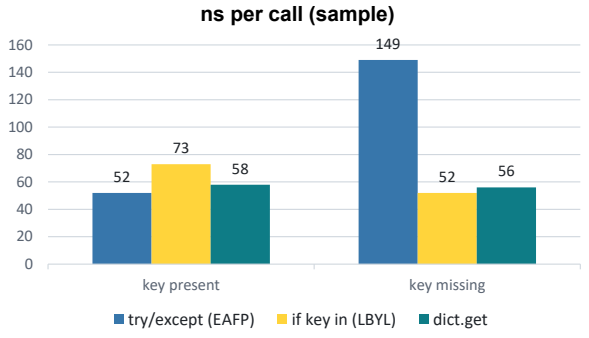
The Cost of Exceptions: EAFP vs LBYL

prices.py

```python
def price_eafp(key: str) -> float | None:
    try:
        return PRICES[key]
    except KeyError:
        return None


def price_lbyl(key: str) -> float | None:
    if key in PRICES:           # two look-ups
        return PRICES[key]
    return None


def price_get(key: str) -> float | None:
    return PRICES.get(key)
```



Key idea

Since 3.11 entering `try` is free; raising and catching costs ~100 ns. EAFP wins when failures are rare.

DISCUSS

Predict the Faster Version

#	Version A	Version B	Situation
1	`self.text += line`	`"".join(lines)`	50,000 short lines
2	`try: d[k]` / `except KeyError`	`d.get(k)`	90 % of the keys are missing
3	`sum([x * x for x in data])`	`sum(x * x for x in data)`	100,000 ints
4	`[square(x) for x in data]`	`list(map(square, data))`	`square` is a `def`

For each pair: A faster, B faster, or about the same? Why?

ANSWER

Predict the Faster Version

#	Answer	Sample timing	Why
1	B, by far	266 ms vs 0.21 ms	the attribute blocks in-place `+=`: quadratic copying
2	B	~149 ns vs ~56 ns per miss	raising and catching costs ~100 ns
3	about the same	4.6 ms vs 4.3 ms	the generator saves ~4 MB of memory, not time
4	about the same	4.9 ms vs 5.9 ms	both call a Python function per item

Key idea

Intuition is a hypothesis. `timeit` is the experiment.

6 · PURE-PYTHON OPTIMIZATIONS

Caching Expensive Calls with lru_cache

parse_log.py

```python
import time
from datetime import date, datetime
from functools import lru_cache


@lru_cache(maxsize=1024)
def parse_day(text: str) -> date:
    return datetime.strptime(text, "%Y-%m-%d").date()


days = [f"2026-{m:02}-{d:02}"
        for m in range(1, 13) for d in range(1, 29)]
log = days * 300            # 100,800 entries, 336 distinct
for parse in (parse_day.__wrapped__, parse_day):
    start = time.perf_counter()
    parsed = [parse(text) for text in log]
    print(f"{time.perf_counter() - start:.3f} s")
print(parse_day.cache_info().hits, "hits")
```

SAMPLE OUTPUT — TIMINGS VARY BY MACHINE

```
0.413 s
0.008 s
100464 hits
```

- Decorator from Module 2: here we measure it
- `__wrapped__` is the undecorated function
- 50× faster: 336 parses instead of 100,800

Watch out

Cache only pure functions with hashable arguments. Callers share the cached result: return immutable values (`date` is immutable).

6 · PURE-PYTHON OPTIMIZATIONS

Saving Memory with __slots__

points.py

```python
class Point:
    def __init__(self, x: float, y: float) -> None:
        self.x, self.y = x, y


class SlottedPoint:
    __slots__ = ("x", "y")

    def __init__(self, x: float, y: float) -> None:
        self.x, self.y = x, y
```

- `__slots__` replaces the per-instance `__dict__` with fixed slots
- `@dataclass(slots=True)` generates it (Module 1)

measure.py

```python
import tracemalloc

for cls in (Point, SlottedPoint):
    tracemalloc.start()
    points = [cls(1.0, 2.0) for _ in range(100_000)]
    size = tracemalloc.get_traced_memory()[0]
    tracemalloc.stop()
    print(f"{cls.__name__:<13}{size / 2**20:4.1f} MiB")
```

SAMPLE OUTPUT — TIMINGS VARY BY MACHINE

```
Point         9.2 MiB
SlottedPoint  5.3 MiB
```

- ~40 % less memory, slightly faster attribute access
- No new attributes: `p.z = 1` → `AttributeError`

6 · PURE-PYTHON OPTIMIZATIONS

Avoid Repeated Work

normalize.py

```python
import time


def normalize_slow(scores: list[float]) -> list[float]:
    return [score / max(scores) for score in scores]


def normalize_fast(scores: list[float]) -> list[float]:
    highest = max(scores)          # computed once
    return [score / highest for score in scores]


scores = [float(i % 97) for i in range(10_000)]
for normalize in (normalize_slow, normalize_fast):
    start = time.perf_counter()
    normalize(scores)
    ms = (time.perf_counter() - start) * 1000
    print(f"{normalize.__name__}: {ms:.2f} ms")
```

SAMPLE OUTPUT — TIMINGS VARY BY MACHINE

```
normalize_slow: 662.12 ms
normalize_fast: 0.45 ms
```

- Hoist loop-invariant work out of the loop
- Here O(n²) → O(n): ~1,400× faster
- Look up once: `value = d.get(k)`
- Compile regular expressions once

Common misconception

Hoisting `len(data)` out of `for i in range(len(data))` saves nothing: `range()` is evaluated once per loop anyway.

6 · PURE-PYTHON OPTIMIZATIONS

Import Time and Deferred Imports

SAMPLE OUTPUT — TIMINGS VARY BY MACHINE (SELF AND CUMULATIVE TIME IN MICROSECONDS)

```
$ python -X importtime -c "import numpy" 2>&1 | tail -3
import time:       575 |      28951 |   numpy.lib
import time:       261 |        261 |   numpy._array_api_info
import time:      1325 |      90758 | numpy
```

report_cli.py

```python
def export_chart(values: list[float], path: str) -> None:
    import matplotlib.pyplot as plt   # loaded on demand

    plt.plot(values)
    plt.savefig(path)
```

- Each module is imported once, then cached in `sys.modules`
- `from json import loads` still loads all of `json`
- Defer heavy imports in CLIs and short-lived scripts
- Otherwise keep imports at the top (PEP 8)

6 · PURE-PYTHON OPTIMIZATIONS

Logging Without Slowing Down

log_cost.py

```python
import logging
import timeit

logging.basicConfig(level=logging.INFO)   # no DEBUG
logger = logging.getLogger("orders")
order = {"id": 42, "items": list(range(20))}


def eager() -> None:
    logger.debug(f"Processing order {order}")


def lazy() -> None:
    logger.debug("Processing order %s", order)


for func in (eager, lazy):
    best = min(timeit.repeat(func, number=100_000))
    print(f"{func.__name__:<6}{best * 1e4:6.0f} ns per call")
```

SAMPLE OUTPUT — TIMINGS VARY BY MACHINE

```
eager   1087 ns per call
lazy     109 ns per call
```

Key idea

Logging calls are the one place where modern code keeps `%s`-style arguments: the message is formatted only if the record is emitted.

- Guard expensive arguments with `logger.isEnabledFor(logging.DEBUG)`
- Do not log inside tight loops
- Slow handlers (files, network): `QueueHandler`

6 · PURE-PYTHON OPTIMIZATIONS

Use with for Files and Other Resources

without with

```python
file = open("orders.txt")
data = file.read()
file.close()
```

with with

```python
from pathlib import Path

path = Path("orders.txt")
path.write_text("A-1\nB-2\n", encoding="utf-8")

with path.open(encoding="utf-8") as file:
    for line in file:          # one line at a time
        print(line.rstrip())
print(file.closed)
```

OUTPUT

```
A-1
B-2
True
```

Key idea

`with` is about **correctness**, not speed: the file is closed even when an exception is raised (Module 4).

- Unclosed files leak descriptors and may lose buffered data
- Iterating over a file uses constant memory
- Always pass `encoding="utf-8"`

TRY IT

Optimise a Slow Report

report.py

```
1  def slow_report(orders: list[tuple[str, float]]) -> str:
2      customers = []
3      text = ""
4      for customer, amount in orders:
5          if customer not in customers:
6              customers.append(customer)
7          share = amount / max(a for _, a in orders)
8          text += f"{customer}: {share:.1%}\n"
9      return text + f"{len(customers)} customers"
```

- Find three performance problems (think: complexity)
- Rewrite it so that 5,000 orders take milliseconds, not a second
- The output must stay identical: prove it with an `assert`

ANSWER

Optimise a Slow Report

```
def fast_report(orders: list[tuple[str, float]]) -> str:
    highest = max(amount for _, amount in orders)      # once
    customers = {customer for customer, _ in orders}   # a set
    lines = [f"{c}: {a / highest:.1%}" for c, a in orders]
    lines.append(f"{len(customers)} customers")
    return "\n".join(lines)                            # join


print(fast_report([("ada", 50.0), ("bob", 100.0), ("ada", 25.0)]))
```

OUTPUT

```
ada: 50.0%
bob: 100.0%
ada: 25.0%
2 customers
```

- Line 5: list membership O(n) → a set
- Line 7: `max` recomputed per order, O(n²) → computed once
- Line 8: repeated `+=` → collect lines, `join` once
- 5,000 orders: 0.93 s → 1.9 ms (sample), identical output

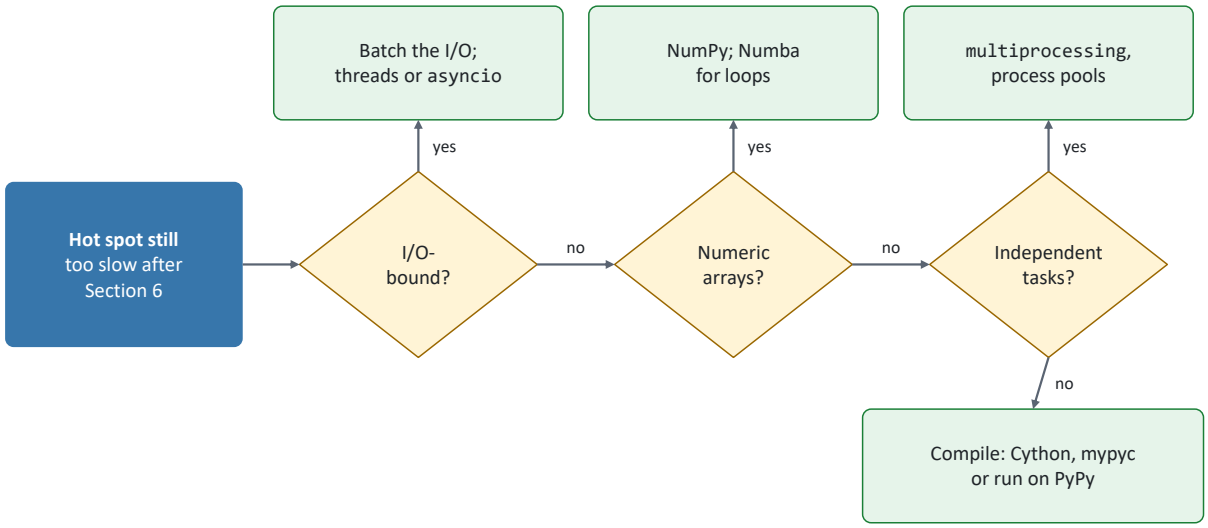
07

Beyond Pure Python

Processes, vectorisation, compilers and batching

- Using all cores with `multiprocessing` (Module 5)
- Vectorising numeric code with NumPy
- Compiling hot loops: Cython, Numba, PyPy, mypyc
- Batching database and file operations

When Pure Python Is Not Enough



Options combine: e.g. a process pool whose workers use NumPy.

Using All Cores with multiprocessing

primes.py

```python
import math
import time
from multiprocessing import Pool

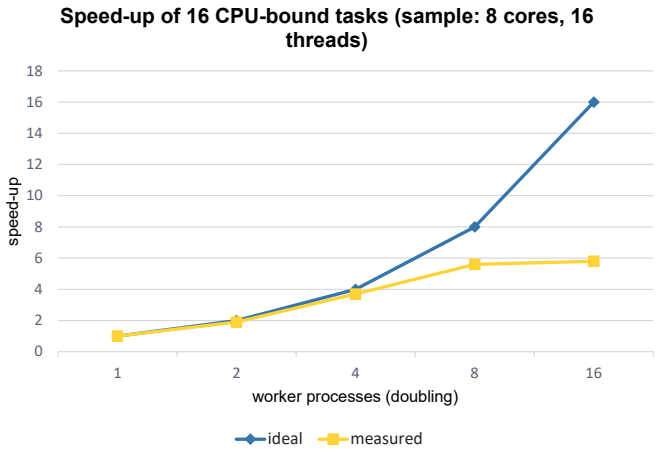

def count_primes(limit: int) -> int:
    """Count primes below limit by trial division (CPU-bound)."""
    return sum(1 for n in range(2, limit)
               if all(n % d for d in range(2, math.isqrt(n) + 1)))


if __name__ == "__main__":           # required with spawn
    limits = [150_000] * 8
    start = time.perf_counter()
    serial = [count_primes(limit) for limit in limits]
    print(f"serial:      {time.perf_counter() - start:.2f} s")
    start = time.perf_counter()
    with Pool(processes=4) as pool:
        parallel = pool.map(count_primes, limits)
    print(f"4 processes: {time.perf_counter() - start:.2f} s")
```

SAMPLE OUTPUT — TIMINGS VARY BY MACHINE

```
serial:      1.55 s
4 processes: 0.65 s
```


When Do Processes Pay Off?



- Each task must do milliseconds of work: the original `Pool(4).map(square, range(10))` took 115 ms, the loop 8.5 µs
- Arguments and results are pickled: send small inputs
- Scaling flattens at the physical cores and by Amdahl's law

Python version note

Free-threaded CPython: 3.13 experimental build, 3.14 officially supported but optional (PEP 779). There, threads can run CPU-bound Python in parallel (Module 5).

7 · BEYOND PURE PYTHON

Vectorising with NumPy

vectorise.py

```python
import timeit

import numpy as np

celsius = [i * 0.01 for i in range(1_000_000)]
celsius_array = np.array(celsius)


def to_fahrenheit_loop() -> list[float]:
    return [c * 9 / 5 + 32 for c in celsius]


def to_fahrenheit_numpy() -> np.ndarray:
    return celsius_array * 9 / 5 + 32


for func in (to_fahrenheit_loop, to_fahrenheit_numpy):
    seconds = min(timeit.repeat(func, number=5)) / 5
    print(f"{func.__name__:<20}{seconds * 1e3:6.1f} ms")
```

SAMPLE OUTPUT — TIMINGS VARY BY MACHINE

```
to_fahrenheit_loop    76.3 ms
to_fahrenheit_numpy    4.5 ms
```

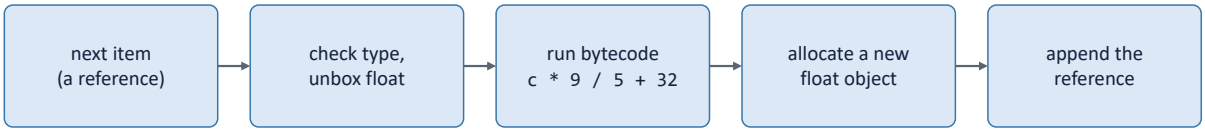
```
>>> import numpy as np
>>> data = np.array([1, 2, 3, 4])
>>> data * 2
array([2, 4, 6, 8])
```

- One expression processes the whole array
- No Python loop: ~17× faster here
- Same result as the comprehension

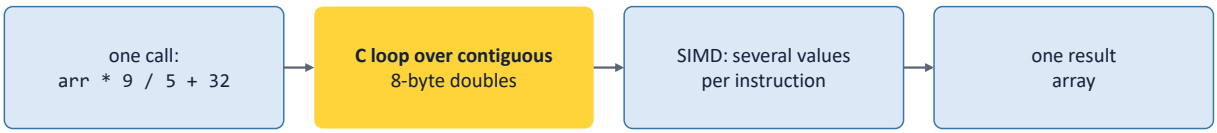
7 · BEYOND PURE PYTHON

Why Vectorised Code Is Fast

Pure Python: these steps run once **per element**, 1,000,000 times



NumPy: the Python interpreter runs **one** expression



Watch out

Looping over a NumPy array **in Python** is slower than over a list (193 ms vs 76 ms, sample): every element is boxed into a new Python object.

Compiling Hot Code: Cython, Numba, PyPy, mypyc

Tool	How it works	Best for	Trade-offs
Cython	compiles Python with C type declarations into a C extension module	hot loops, wrapping C libraries	build step and C compiler
Numba	JIT-compiles decorated functions to machine code (LLVM) at the first call	numeric loops over NumPy arrays	subset of Python; compile delay
PyPy	alternative Python interpreter with a tracing JIT compiler	long-running pure-Python programs	slower C extensions; lags behind CPython versions
mypyc	compiles type-annotated modules into C extensions	typed pure-Python libraries	annotations must be correct; build step

Python version note

CPython itself keeps getting faster: the specialising interpreter in 3.11 (~25 % on average) and an experimental JIT since 3.13. Upgrading is the cheapest optimisation.

Numba: JIT-Compiling a Loop

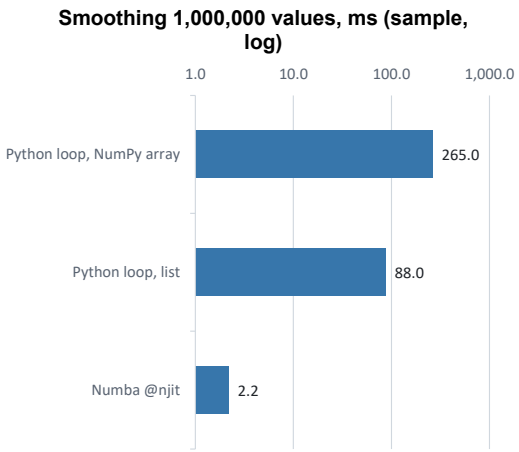
smooth.py

```python
import numpy as np
from numba import njit


@njit
def smooth(values: np.ndarray, alpha: float) -> np.ndarray:
    """Exponential moving average: each step needs the last."""
    result = np.empty_like(values)
    result[0] = values[0]
    for i in range(1, len(values)):
        result[i] = (alpha * values[i]
                     + (1 - alpha) * result[i - 1])
    return result
```

Note

The first call compiles (~0.5 s here); later calls run at machine-code speed.



7 · BEYOND PURE PYTHON

Type Hints Do Not Speed Up CPython

hints.py

```python
import timeit


def area(width, height):
    return width * height


def typed_area(width: float, height: float) -> float:
    return width * height


for func in (area, typed_area):
    runs = timeit.repeat(lambda: func(3.0, 4.0), number=1_000_000)
    print(f"{func.__name__:<11}{min(runs) * 1e3:5.1f} ns per call")
print(typed_area("ab", 3))          # not checked at run time
```

SAMPLE OUTPUT — TIMINGS VARY BY MACHINE

```
area        51.2 ns per call
typed_area  50.0 ns per call
ababab
```

- CPython stores annotations but does not use them to optimise
- Benefits: readability, IDE help, `mypy` / `pyright` (Module 1)
- mypyc and Cython *can* use them to compile faster code

✗ **Common misconception**

"Add type hints for speed" is a myth for CPython: they are documentation that tools can check.

QUIZ

Quick Check: Performance Myths

Which statement is **true**?

A. Adding type hints makes CPython execute a function faster

B. Iterating over a NumPy array in a Python `for` loop is faster than over a list

C. A process pool can make a program slower when each task is tiny

D. PyPy runs every C-extension library faster than CPython

ANSWER

Quick Check: Performance Myths

Which statement is **true**?

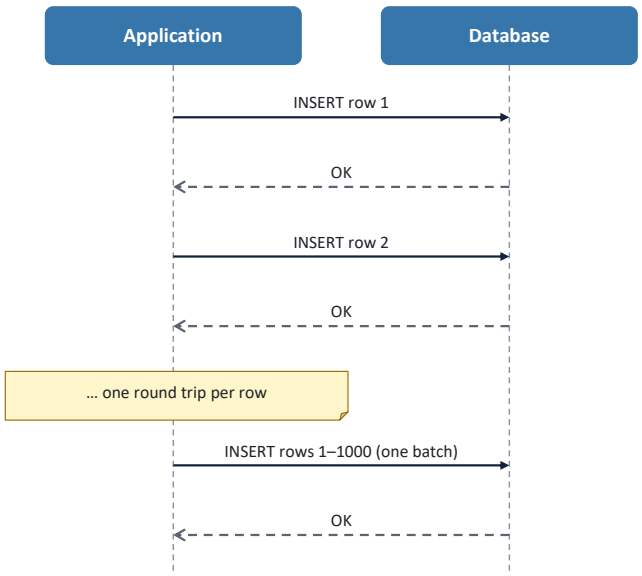
- A — Adding type hints makes CPython execute a function faster
- B — Iterating over a NumPy array in a Python `for` loop is faster than over a list
- **C — A process pool can make a program slower when each task is tiny**
- D — PyPy runs every C-extension library faster than CPython

Why

Starting worker processes and pickling arguments and results cost far more than a tiny task: the original `Pool(4).map(square, range(10))` took 115 ms instead of 8.5 µs.

7 · BEYOND PURE PYTHON

Batch Processing: Amortise the Overhead



n × (overhead + work) → (n / b) × overhead + n × work

- Database writes: bulk inserts and updates
- Files and networks: fewer, larger operations
- Machine learning: mini-batches of examples

Key idea

Batching trades latency for throughput (Section 1): an item waits until its batch is sent.

Batch Database Writes

batch_sqlite.py

```python
import sqlite3
import time

rows = [(i, f"s-{i % 10}", i * 0.5) for i in range(500)]
sql = "INSERT INTO readings VALUES (?, ?, ?)"
conn = sqlite3.connect("readings.db")
conn.execute("CREATE TABLE readings (id, sensor, value)")

start = time.perf_counter()
for row in rows:
    conn.execute(sql, row)
    conn.commit()               # a transaction per row
print(f"one by one: {time.perf_counter() - start:.3f} s")

start = time.perf_counter()
with conn:                      # one transaction
    conn.executemany(sql, rows)
print(f"batch:      {time.perf_counter() - start:.3f} s")
conn.close()
```

SAMPLE OUTPUT — TIMINGS VARY BY MACHINE

```
one by one: 1.476 s
batch:      0.004 s
```

PyMongo (Python) — Module 7

```python
from pymongo import InsertOne, UpdateOne

readings.insert_many(documents, ordered=False)

readings.bulk_write([
    InsertOne({"sensor": "s-1", "value": 20.5}),
    UpdateOne({"sensor": "s-2"},
              {"$set": {"value": 21.0}}),
])
```

Tip

Also: reuse connections (connection pools) and index the fields you query (Module 7).

Batch File Processing

batch_file.py

```python
from itertools import batched
from pathlib import Path

path = Path("events.txt")
lines = (f"event {i}\n" for i in range(2_500))
path.write_text("".join(lines), encoding="utf-8")


def save(batch: tuple[str, ...]) -> None:
    print(f"saving {len(batch)} lines from {batch[0].strip()!r}")


with path.open(encoding="utf-8") as file:
    for batch in batched(file, 1_000):    # Python 3.12+
        save(batch)
```

OUTPUT

```
saving 1000 lines from 'event 0'
saving 1000 lines from 'event 1000'
saving 500 lines from 'event 2000'
```

Common misconception

`for line in file` is not slow: file objects are already buffered. Batch when the **processing** has per-call overhead (a database insert, an HTTP call, a NumPy operation).

- `itertools.batched` replaces the manual list-and-flush loop
- Huge binary files: `file.read(1 << 20)` chunks or `mmap`
- Bounded batches keep memory bounded

QUICK CHECK

Choose the Right Technique

#	Situation
1	Resize 5,000 photos with pure-Python code on an 8-core machine
2	Standardise 50 million floats: `(x - mean) / std`
3	A loop over 10 million floats where each step needs the previous result
4	Import 2 million CSV rows into MongoDB
5	Call 200 slow web APIs and combine the answers
6	Speed up a large pure-Python application you cannot rewrite

ANSWER

Choose the Right Technique

#	Technique	Why
1	`ProcessPoolExecutor` / `multiprocessing`	CPU-bound, independent tasks
2	NumPy vectorisation	one whole-array expression
3	Numba `@njit` (or Cython)	sequential dependency: cannot vectorise
4	batches: `insert_many` with 1,000–10,000 documents	per-request overhead
5	threads or `asyncio` (Module 5)	I/O-bound: waiting, not computing
6	try PyPy (check the C extensions)	a JIT without code changes

MODULE 11 · PRACTICE

Practice Problems

1. Profile and Fix

- Profile the billing script from the quiz
- Find the hot spot with `cProfile`, then `line_profiler`
- Prove a 20× gain with `pytest-benchmark`

2. Memory Diet

- Store 1,000,000 sensor readings as dicts, `__slots__` objects, tuples and a NumPy array
- Compare with `tracemalloc`

3. Scaling Curve

- Count primes with 1–16 processes
- Plot the speed-up
- Estimate the serial fraction with Amdahl's law

4. Batch Size Sweep

- Insert 100,000 rows into SQLite or MongoDB
- Batch sizes 1, 10, 100, 1,000, 10,000
- Plot rows per second

MODULE 11 · SUMMARY

Key Takeaways

- ✓ Define measurable goals first: latency, throughput, utilisation, capacity (Sections 1–2)
- ✓ Amdahl's law: the part you do not speed up limits the total gain
- ✓ Monitor the system top-down with OS tools before changing code (Sections 3–4)
- ✓ Benchmark with `timeit` and `pytest-benchmark`; profile with `cProfile`, `line_profiler`, `tracemalloc`, `py-spy`
- ✓ Fix algorithms and data structures first, then hot loops: built-ins, caching, `__slots__`
- ✓ Beyond pure Python: processes for CPU-bound work, NumPy and Numba for numbers, batching for I/O
- ✓ Measure again after every change — and keep the code correct, tested and readable

NEXT UP Course wrap-up — final project and exam review